



迴歸、聚類、神經網路

劉益宏

國立陽明交通大學電控所教授

1

迴歸

Regression

2

聚類

Clustering

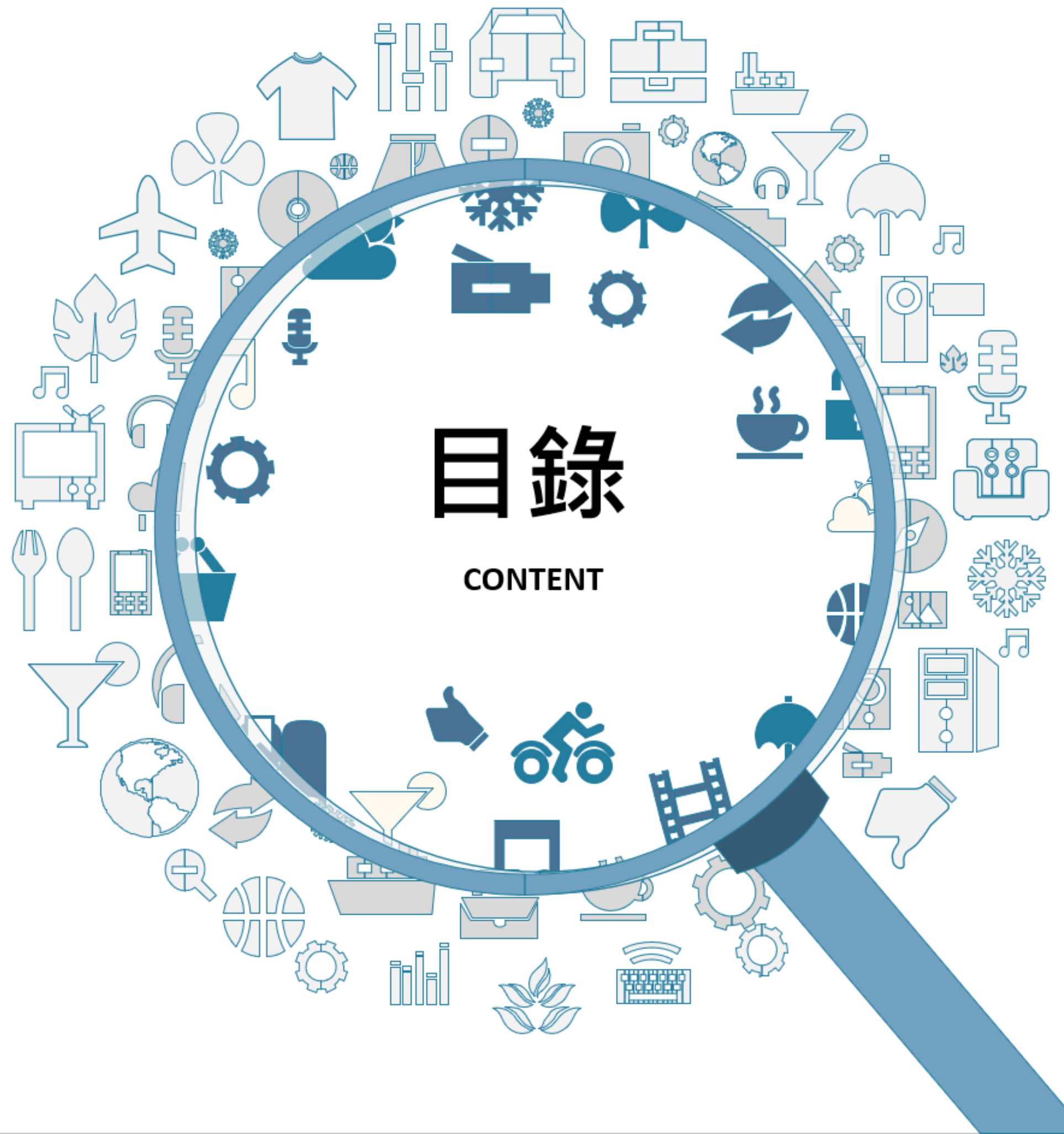
3

神經網路

Neural networks

目錄

CONTENT





NessLab

1. 迴歸與分類

- 在監督式學習中，**分類 (Classification)** 和 **回歸 (Regression)** 是針對不同應用場景而使用的不同方法。主要差異在於預測的結果類型。

當你預測的目標輸出為離散的數值，稱其為分類(例如：判斷郵件是垃圾還是非垃圾)。

若為連續的數值，我們稱其為迴歸(例如：預測股票價格或學生成績)。

$$\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \text{variable/feature}$$

$$f(\vec{x}) = y \text{ 值 (預測值)}$$

$\vec{x} \rightarrow \boxed{f(\cdot)} \rightarrow y$

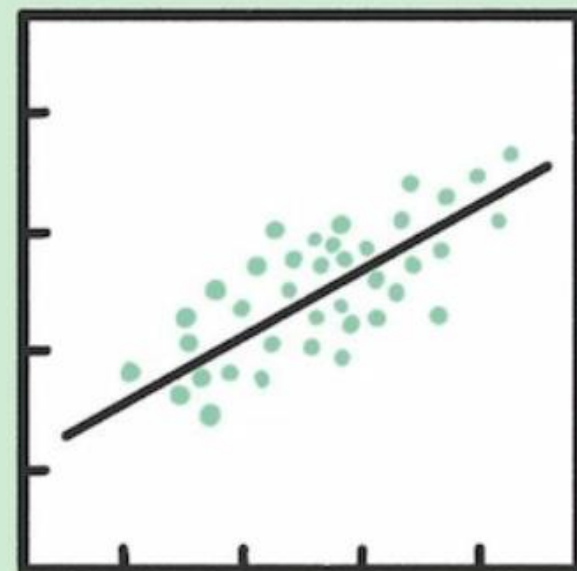
Regression vs. 預測

Given: $\vec{y}$ 實際值

↓ error 誤差

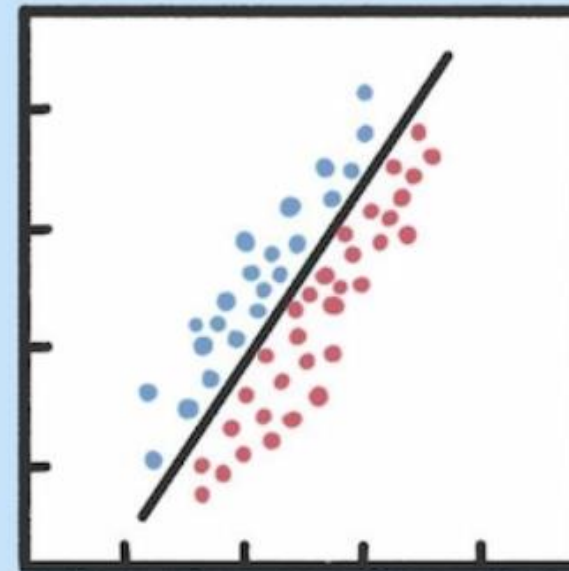
找到資料分布規則的函式

迴歸問題



在迴歸問題中，
找到符合資料分析的函式

分類問題



在分類問題中，
找到將不同類別資料分開的函式

找到分開資料的函式



1.迴歸與分類

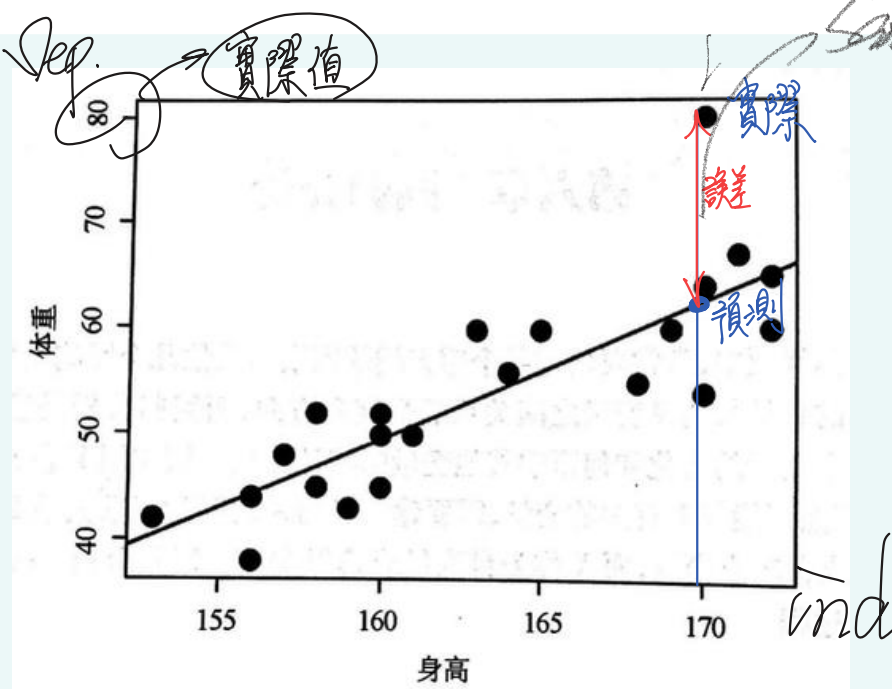
• 迴歸的精神：

1. 旨在理解和量化**自變數** (Independent variable) 與**應變數** (Dependent variable) 之間的關係。

只有一個**自變數**和一個**應變數**的情形稱為**簡單線性迴歸**(Simple linear regression) ,

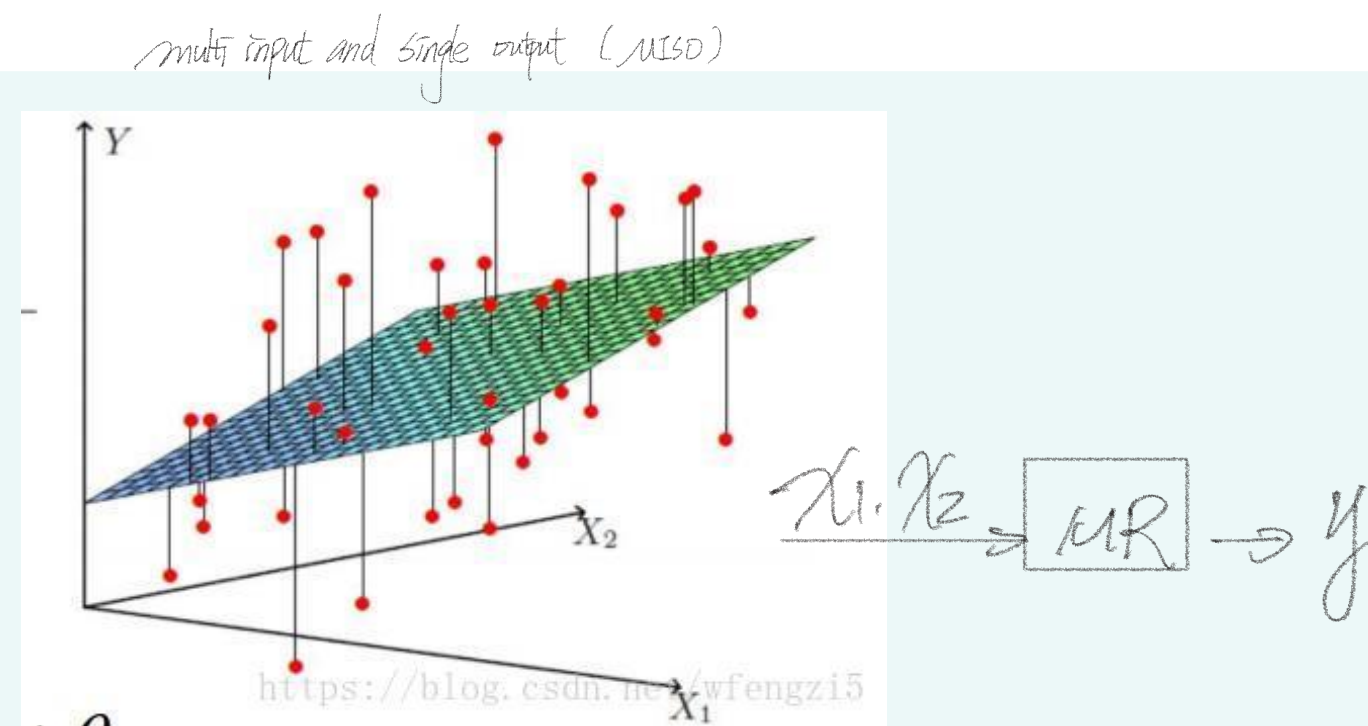
大於一個**自變數**的情形稱為**多元迴歸** (Multiple regression) 。

簡單線性回歸：



假設自變數=身高= x ，應變數=體重= y

二元線性回歸：



有兩個自變數 x_1 、 x_2

2. 迴歸模型能夠捕捉變數之間的**趨勢**和**關聯性**。

★兩個都是 linear



NessLab

2. 簡單線性迴歸 (Simple linear regression)

$\Rightarrow$ Single input single output (SISO)

• 實例推導：

假設有 n 筆資料 $(x_i, y_i), i = 1, \dots, n$ ，橫軸為身高，縱軸體重，預求這兩變數所形成的迴歸方程式。

$\hat{f}(x_i) = \hat{y}_i$: 預測值

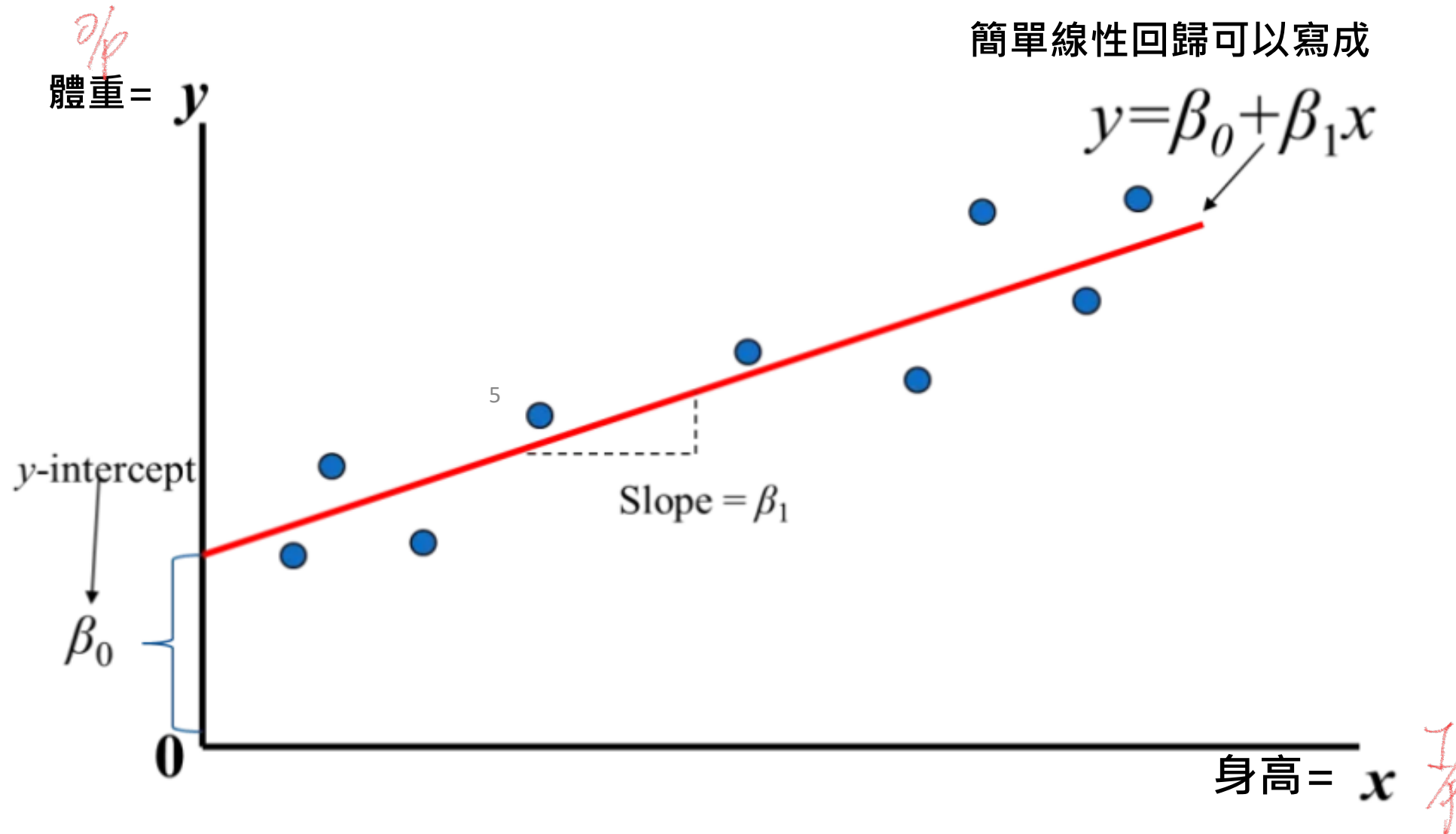
$\hat{f}(x_n) = \hat{y}_n$:

整體 error

$(\hat{y}_1 - y_1) + \dots + (\hat{y}_n - y_n)$

$\Rightarrow (\hat{y}_1 - y_1)^2 + \dots + (\hat{y}_n - y_n)^2$ 保證誤差為正

$\Rightarrow \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$ mean square error (MSE)



β_0 : 截距(Intercept)

β_1 : 斜率(Slope)

n : 資料總數



NessLab

2. 簡單線性迴歸 (Simple linear regression)

β_0 : 截距(Intercept)

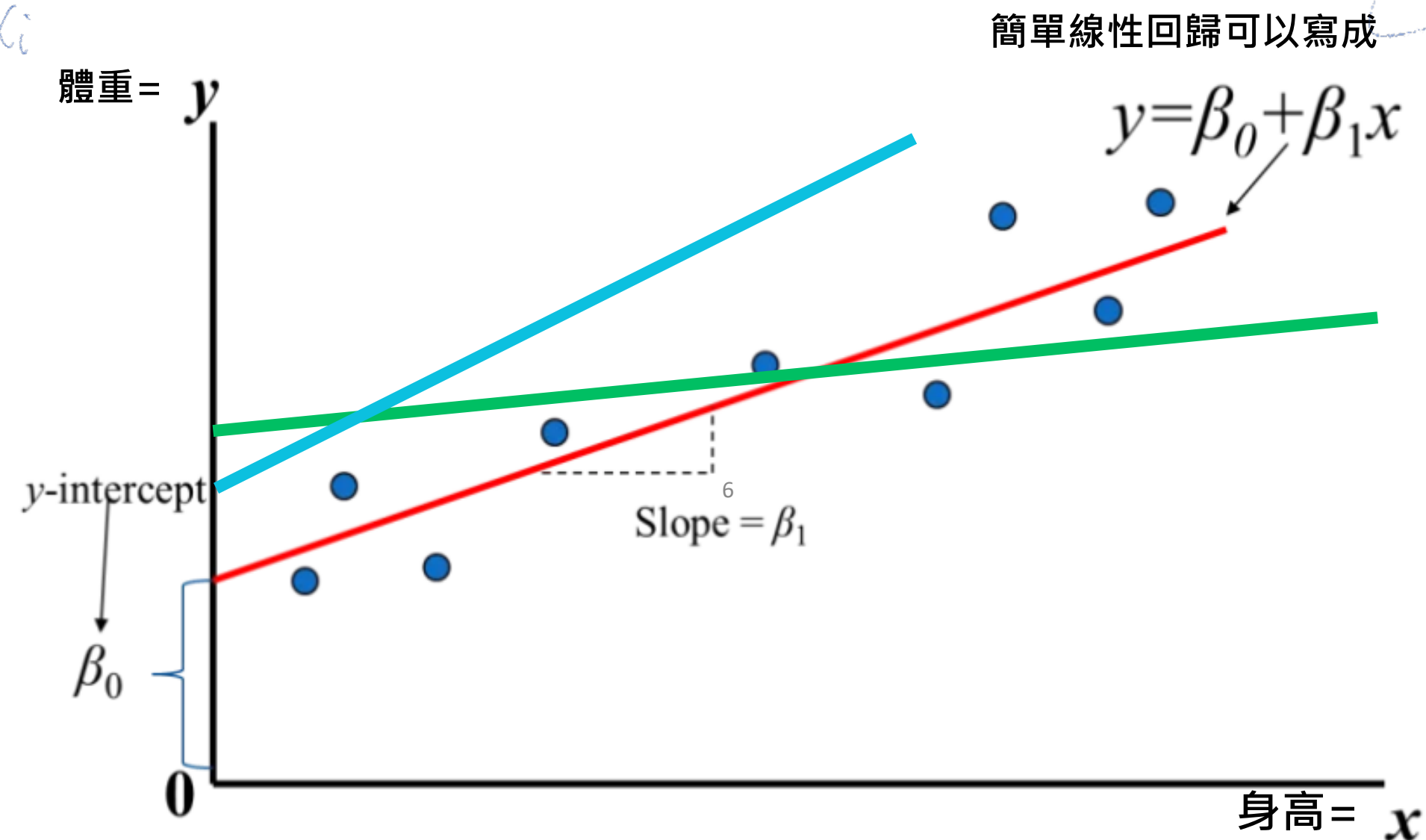
β_1 : 斜率(Slope)

n : 資料總數

- 但這迴歸方程式有很多條，如何找到最好、最有代表性的迴歸方程呢？

$\rightarrow \min MSE$

$$y_i = \beta_0 + \beta_1 x_i$$



也就是如何找到最好的 β_0 、 β_1 ?



2. 簡單線性迴歸(Simple linear regression)

- 先定義 $\hat{y}_i$ ：將 n 個資料點, $(x_i, y_i), i = 1, \dots, n$ 都代入迴歸方程式內，是為預估的體重。

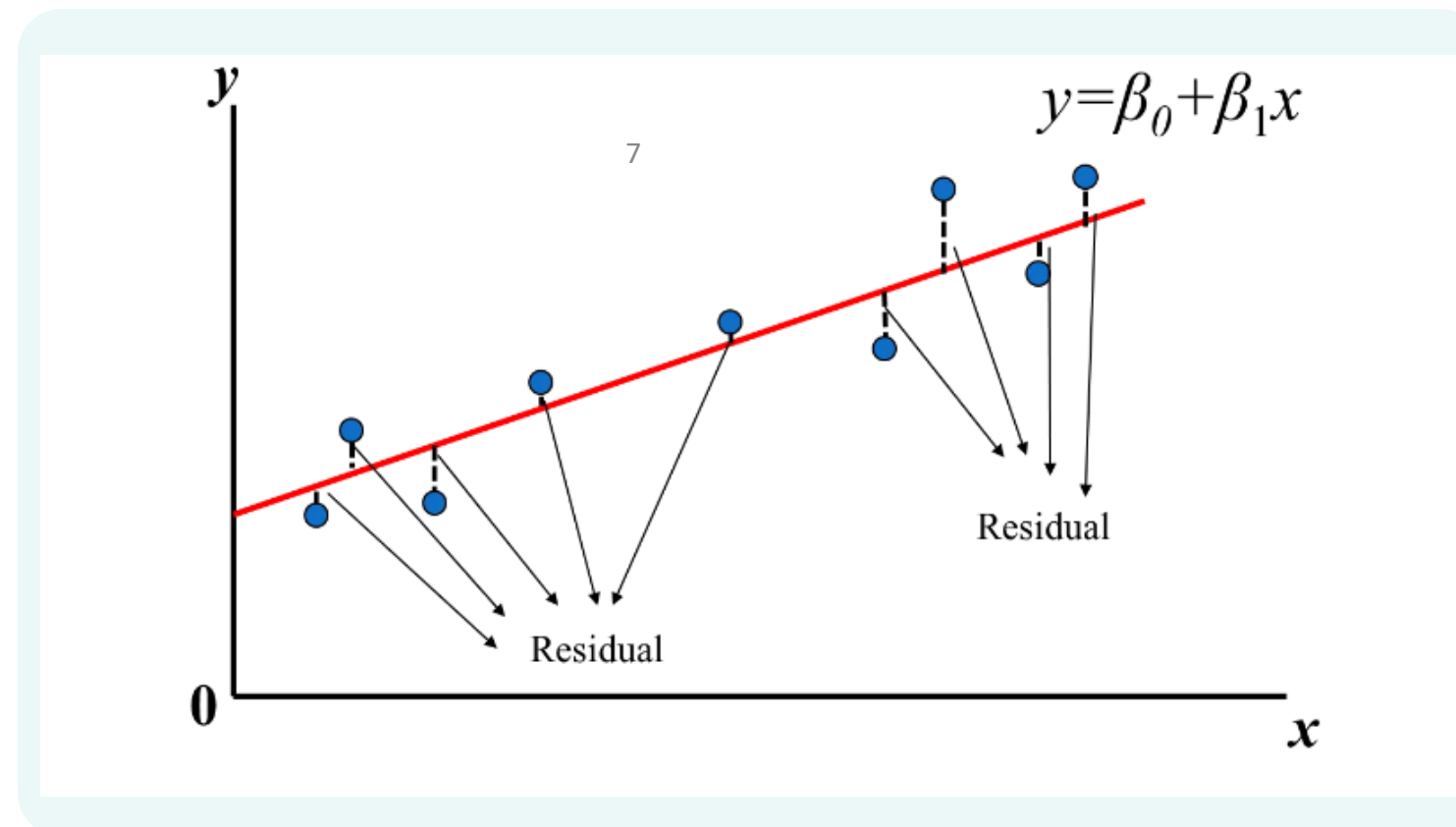
predicted value

$$\hat{y}_i = \beta_0 + \beta_1 x_i, i = 1, \dots, n$$

- 再定義 ε_i ：預估出來的體重和真實資料還是會有誤差(error)或稱為殘差(Residual)。

$$\varepsilon_i = y_i - \hat{y}_i$$

actual value



單輸入單輸出

f(x_i) = \hat{y}_i prediction

if f is linear then, \hat{y}_i = \beta_1 x_i + \beta_0

← 迴歸分析出 \beta_0, \beta_1



2. 簡單線性迴歸 (Simple linear regression)

- 迴歸分析的目標函數/損失函數(loss function)就是希望找到的模型最終的殘差越小越好，統計上會用一個名詞：最小平方法(Least Square)來找參數(β_0 、 β_1)。

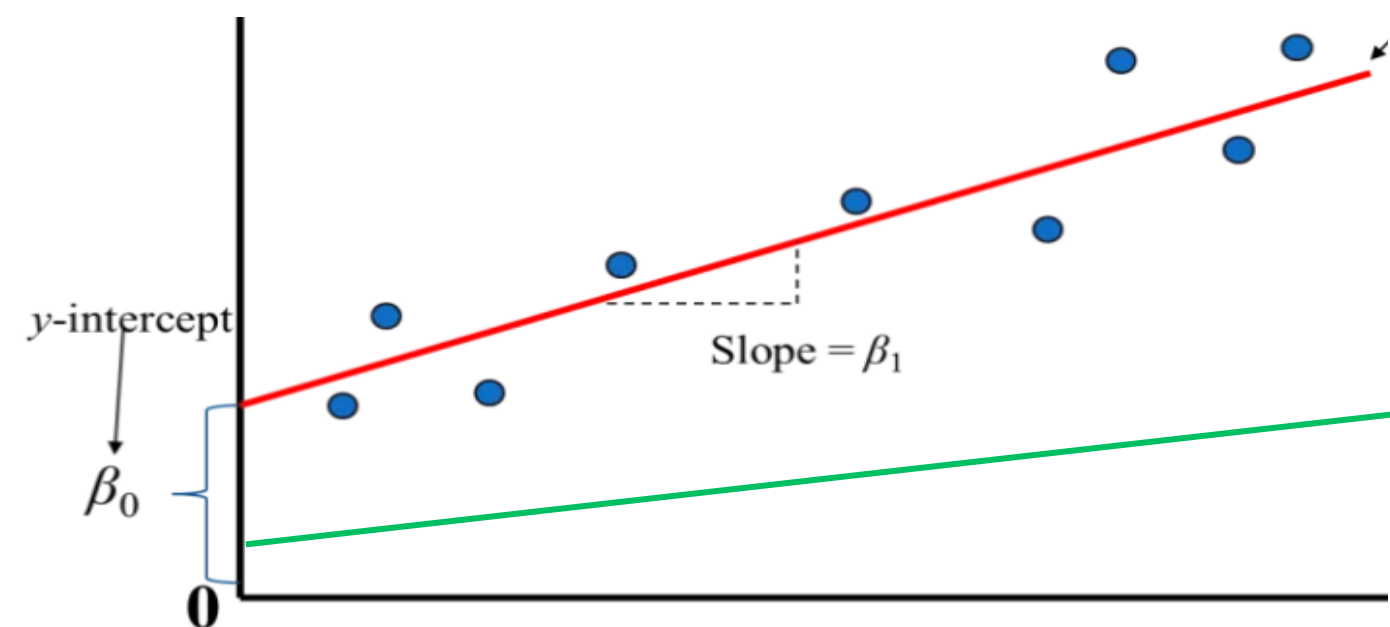
$$\text{loss}(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (\text{known } y_i - (\text{unknown } \hat{\beta}_0 + \text{known } \hat{\beta}_1 x_i))^2$$

Loss 是 β_0, β_1 的函數

$\beta_0 + \beta_1 x_i$

因此我們希望訓練樣本的誤差平方和(Sum Square error, SSE)越小越好。

$$\textcircled{1} \quad \frac{\partial L}{\partial \beta_0} = 0$$
$$\textcircled{2} \quad \frac{\partial L}{\partial \beta_1} = 0$$



Good

Poor



2. 簡單線性迴歸 (Simple linear regression)

NessLab

- 為了推估 β_0 ，對 $loss(\beta_0, \beta_1)$ 做 β_0 偏微分等於0

$$\begin{aligned} loss(\hat{\beta}_0, \hat{\beta}_1) \\ = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i))^2 \end{aligned}$$

$$\frac{\partial loss(\hat{\beta}_0, \hat{\beta}_1)}{\partial \beta_0} = \frac{\partial \sum_{i=1}^n (y_i - \hat{\beta}_0 + \hat{\beta}_1 x_i)^2}{\partial \beta_0} = 0$$

$$\rightarrow -2 \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0$$

$$\rightarrow \hat{\beta}_0 = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{\beta}_1 x_i)$$

$$\rightarrow \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Handwritten notes: $\bar{y}$ is the average of y_i , $\bar{x}$ is the average of x_i . A red circle with the number 1 is next to the equation.



2. 簡單線性迴歸 (Simple linear regression)

NessLab

- 為了推估 β_1 ，對 $loss(\beta_0, \beta_1)$ 做 β_1 偏微分等於0

$$\frac{\partial loss(\hat{\beta}_0, \hat{\beta}_1)}{\partial \beta_1} = \frac{\partial \sum_{i=1}^n (y_i - \hat{\beta}_0 + \hat{\beta}_1 x_i)^2}{\partial \beta_1} = 0$$

$$loss(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i))^2$$

$$\rightarrow -2 \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) x_i = 0$$

$$\rightarrow \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) x_i = 0$$

$$\rightarrow \sum_{i=1}^n y_i x_i - \sum_{i=1}^n \hat{\beta}_1 x_i^2 - \sum_{i=1}^n \hat{\beta}_0 x_i = 0 \quad \text{1}$$

$$\rightarrow \sum_{i=1}^n y_i x_i - \hat{\beta}_1 \sum_{i=1}^n x_i^2 - \sum_{i=1}^n (\bar{y} - \hat{\beta}_1 \bar{x}) x_i = 0$$

$$\rightarrow \sum_{i=1}^n y_i x_i - \hat{\beta}_1 \sum_{i=1}^n x_i^2 - \sum_{i=1}^n \bar{y} x_i + \sum_{i=1}^n \hat{\beta}_1 \bar{x} x_i = 0$$



2. 簡單線性迴歸 (Simple linear regression)

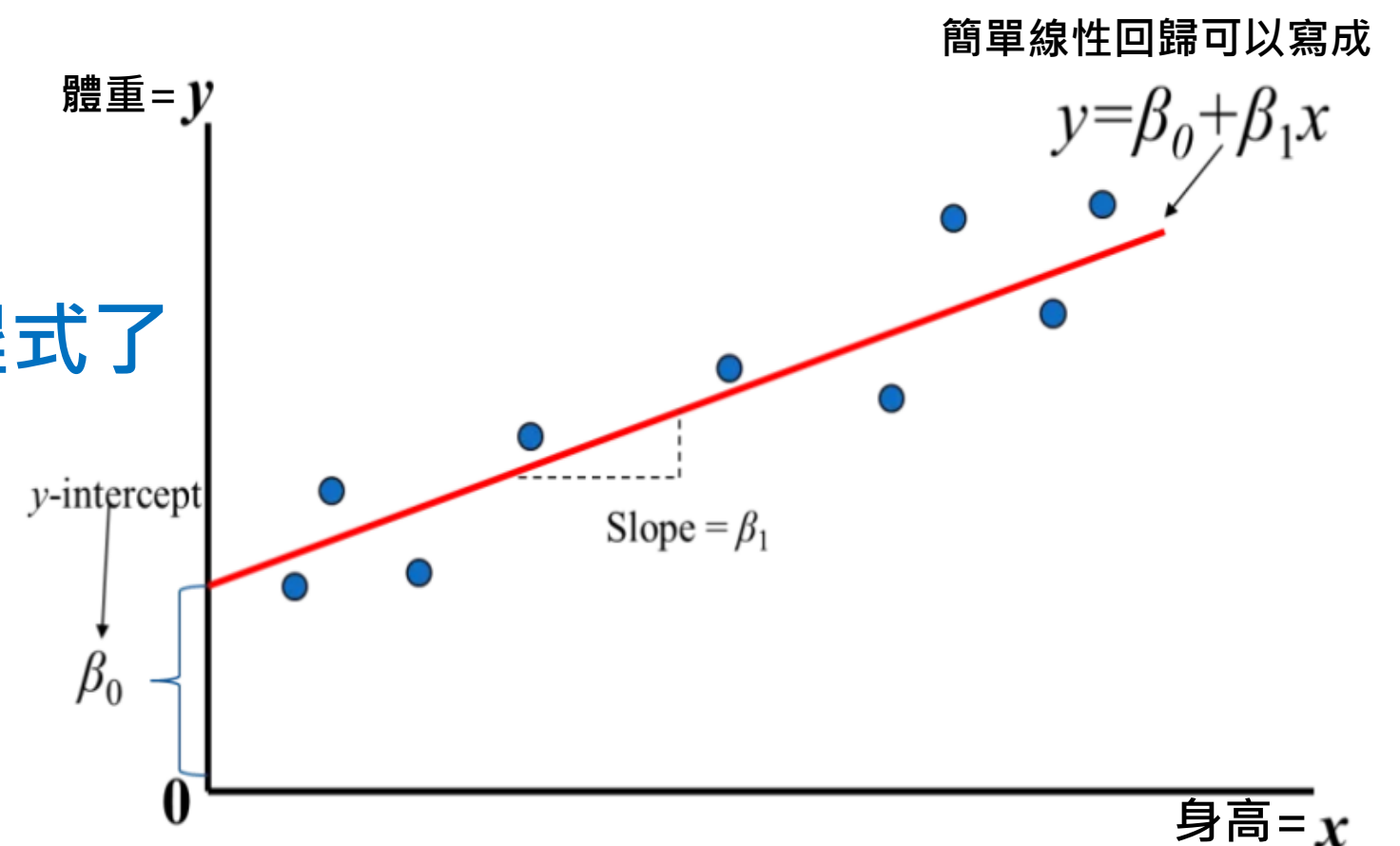
$$\rightarrow \sum_{i=1}^n y_i x_i - \hat{\beta}_1 \sum_{i=1}^n x_i^2 - \sum_{i=1}^n \bar{y} x_i + \sum_{i=1}^n \hat{\beta}_1 \bar{x} x_i = 0$$

$$\rightarrow \hat{\beta}_1 \sum_{i=1}^n (x_i - \bar{x}) x_i = \sum_{i=1}^n (y_i - \bar{y}) x_i$$

$$\rightarrow \hat{\beta}_1 = \frac{\sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

11

- 求得 β_0 、 β_1 就可以推出這條簡單線性迴歸的方程式了



Regression model:

① Linear

- Simple regression (SISD)
- multiple regression (MISO)

② Nonlinear

- (i) Support Vector Regression (SVR) 支持向量回歸。
⇒ 針對樣本數不多的 model.

- (ii) Ridge Regression (RR)

⇒ 介由 polynomial fit 去 regression
多項式擬合

degree 可為 1, 2, 3 ... → 當為 1 時, 為 linear
為 2, 3 ... 時, 為 nonlinear.

- (iii) LASSO (RR 的 variant)

- (iv) random forest (RF): 由 many decision tree 構成

- (v) Neural Network



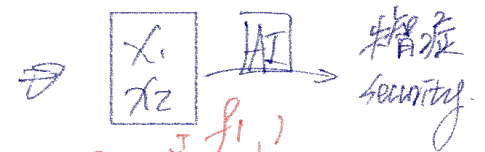
NessLab

3. 多元線性迴歸 (Multiple linear regression)

β_0 : 截距(Intercept)
 $\beta_1 \dots \beta_k$: 斜率(Slope)
 n : 資料總數

- 多元線性迴歸跟前面簡單線性迴歸基本上一樣，只是多了幾個自變數，因此多元迴歸的自變數(x)，可以用矩陣的方式(x)來推導，基本上和簡單線性迴歸大同小異。

ex. x_1 = 蛋白質濃度
 x_2 = brain wave 連結性強度

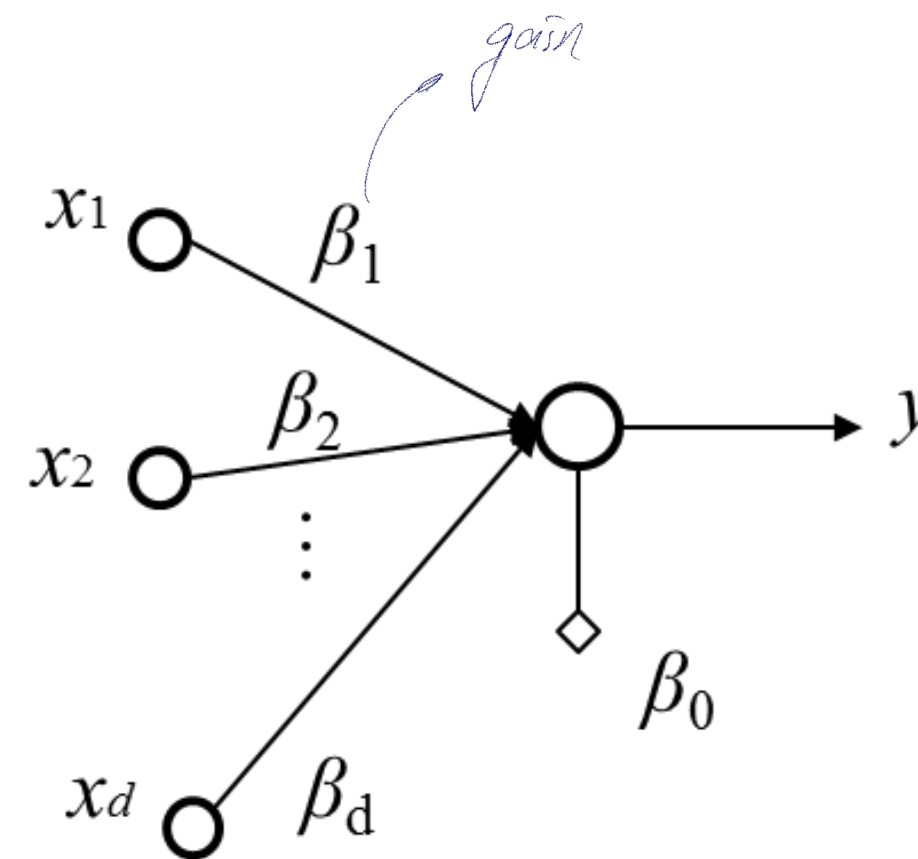


linear regression
 nonlinear

線性多元回歸 $y = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$

多 variable 1st. feature kth feature

$(\beta_0, \beta_1, \dots, \beta_k)$ 參數集
 y : 預測值



12

- 這結構似乎很像只有一個輸入層和一個輸出層的神經網路，所以可以用SGD去找 β 的最佳解。

隨機梯度下降法(Stochastic gradient descent, SGD)

待補多元現行回歸的方程式



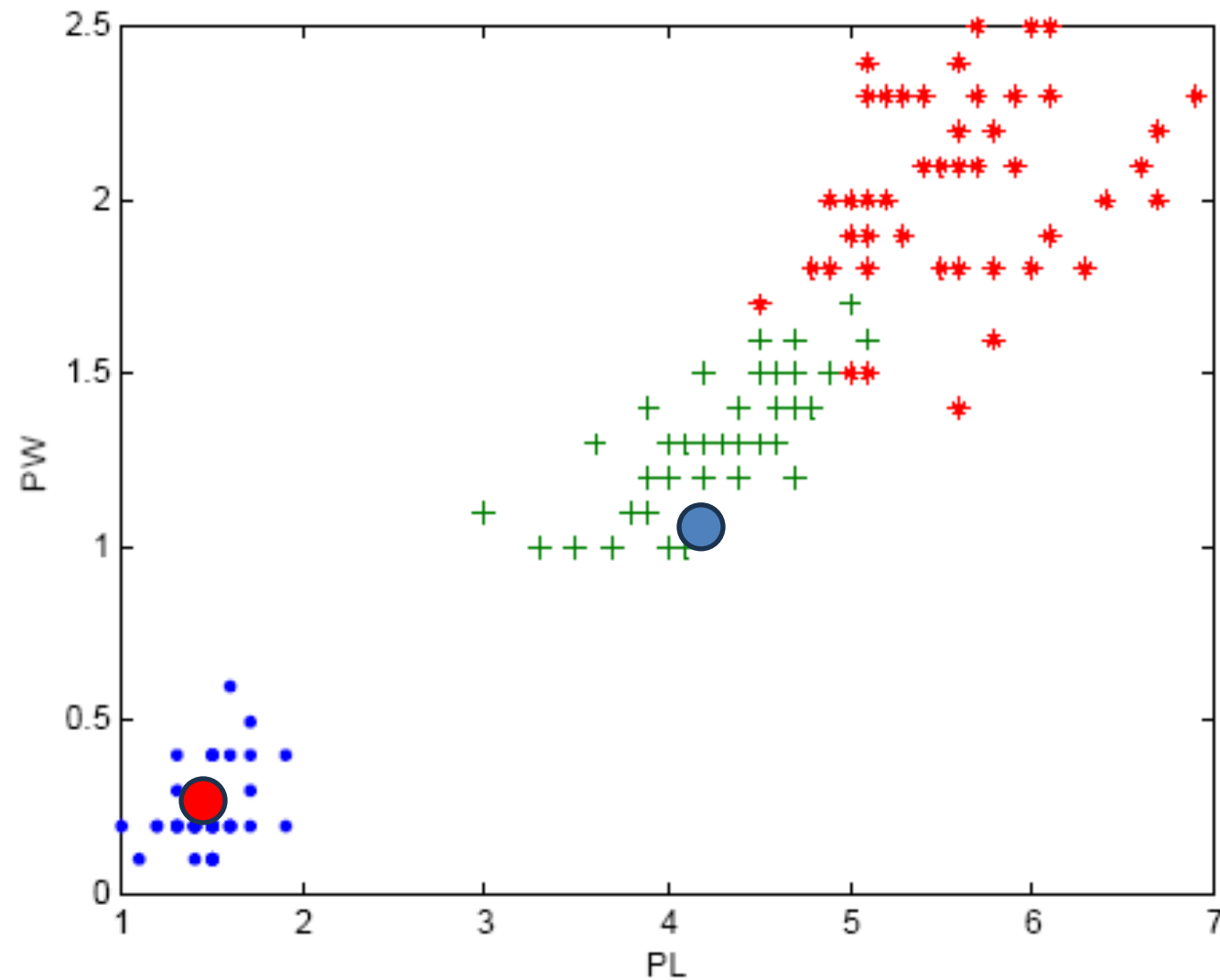
聚類 Clustering

NessLab

大考試不考怎計算

Unsupervised Learning Algorithm

- Basic concept : 以IRIS DATA為例說明



K=2 可以分成兩群 data

if 未知則 how ?

The symbols “.”, “+”, and “*” represent the classes *Iris setosa* (class 1), *Iris versicolor* (class 2), and *Iris virginica* (class 3), respectively



NessLab

Clustering algorithm : K-means algorithm

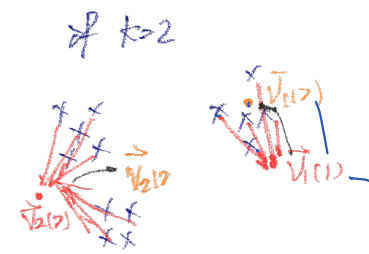
平均
苛薄

↓ 要先定義共有k群 = 算法的參數 user defined.

K-means algorithm

- Given a set of unlabeled data points ($X_1, X_2, \dots, X_n$)
- Initialize the number of clusters k (to cluster the data points into k groups), where $k = 2, 3, \dots, n$. (a user-defined parameter)
- **Step 1** : initialize the centers of the k clusters randomly (利用亂數產生k個座標，當作這k個clusters的初始中心座標)
- **Step 2** : assign the data points to the clusters based on their distance to the cluster centers (把每一筆data歸類到其中的任何一個cluster/group，根據min distance criterion)
- **Step 3**: re-compute the centroids (centers) of the k clusters
- **Step 4**: go to 2nd step until convergence (新的cluster centers 和上一次的cluster centers不再有所差距)

Convergence: 直到 $\sum_{i=1}^n \|V_i(t) - V_i(t-1)\| \leq \epsilon$
Where ϵ 是足夠小



先隨機生成k個中心點
根據最小距離決定cluster分群
再根據分群的群心來決定新的中心
⇒ 但重新定中心後，可能會導致cluster變化
⇒ 不斷重複直到兩個中心點距離變化不大



NessLab

K-means Algorithm - discussion

- Result can vary significantly based on initial choice of seeds
- Algorithm can get trapped in a local minimum
 - ◆ Example : four data points at the vertices of a two-dimensional rectangle
 - Local minimum : two cluster centers at the midpoints of the rectangles long sides
- Simple way to increase chance of finding a global optimum : restart with different random seeds

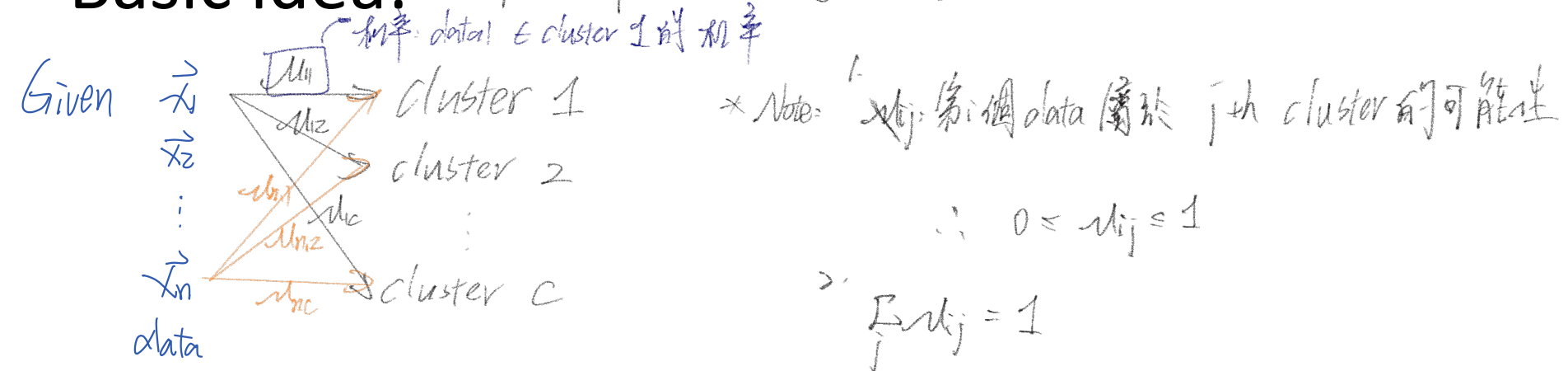


模糊

Fuzzy c-means (FCM) algorithm:

NessLab another data clustering method

- Basic idea: if define there S c clusters



求出所有的 $u_{ij}, i=1 \dots n, j=1 \dots c$



Fuzzy c-means (FCM) algorithm: formulation

- Given a dataset $\{\mathbf{x}_i \in \mathbf{R}^d\}_{i=1}^N$, (FCM) algorithm is to partition the dataset into disjoint clusters by solving the optimization problem,

where

$$\text{Minimize } J(T; \mathbf{U}; \mathbf{V}) = \sum_{j=1}^c \sum_{i=1}^N u_{ji}^m \|\mathbf{x}_i - \mathbf{v}_j\|^2$$

subject to the constraints,

$$\begin{aligned} \sum_{j=1}^c u_{ji} &= 1, \quad \forall i = 1, \dots, N, \\ 0 < \sum_{i=1}^N u_{ji} &< N, \quad \forall j = 1, \dots, c \end{aligned}$$

- $u_{ji} \in [0,1]$ is the membership degree that the i th data point $\mathbf{x}_i$ belongs to the j th cluster
- $\mathbf{U} = [u_{ji}]$ is the partition matrix
- $\mathbf{v}_j \in \mathbf{R}^d$ is the centroid of the j th cluster
- $\mathbf{V} = (\mathbf{v}_1, \dots, \mathbf{v}_j, \dots, \mathbf{v}_c)$ is a c -tuple of cluster prototypes
- c is the number of clusters ($2 \leq c \leq N$)
- $m \in (1, \infty)$ is the weight controlling the degree of fuzziness of the matrix $\mathbf{U}$ (m is set as 2 in general)

例: $m=2$ $m=10$
 $\Rightarrow m^{10} \sim 10$
 $\Rightarrow$ 近似於 k-mean.

* 常取 $m=2$
 $\Rightarrow$ 通過連續

m 越大 模糊程度
 越大

也須要自訂, 但常見為 1, 2.



Fuzzy c-means (FCM) algorithm

– learning steps

- Step 1 : *initialize the number of clusters c*
- Step 2 : *initialize $\mathbf{v}_j$ by random*
- Step 3 : *compute u_{ji} by $u_{ji} = \frac{(1/\|\mathbf{x}_i - \mathbf{v}_j\|^2)^{1/(m-1)}}{\sum_{k=1}^c (1/\|\mathbf{x}_i - \mathbf{v}_k\|^2)^{1/(m-1)}}$*
- Step 4 : *update the cluster centroids $\mathbf{v}_j$ by $\mathbf{v}_j = \frac{\sum_{i=1}^N u_{ji}^m \mathbf{x}_i}{\sum_{i=1}^N u_{ji}^m}$, $j = 1, \dots, c$*
- Step 5 : *repeat step 3 and step 4 until $\|\mathbf{U}\| < \varepsilon$, where ε is a small positive value.*



Fuzzy c-means (FCM) algorithm

– to determine the optimal (most plausible) number of clusters

NessLab

- Choosing a proper number c of clusters is of primary importance. The best number c_{best} of clusters can be determined using the partitioning entropy (分割熵)-based criterion

$$c_{best} = \arg \min_{c=2}^N \{ \min_{\mathbf{U} \in \Omega_c} [H(\mathbf{U}, c)] \}$$

where Ω_c is the set of solutions, and $H(\mathbf{U}, c) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^c |u_{ji} \ln u_{ji}|$

entropy 分割熵

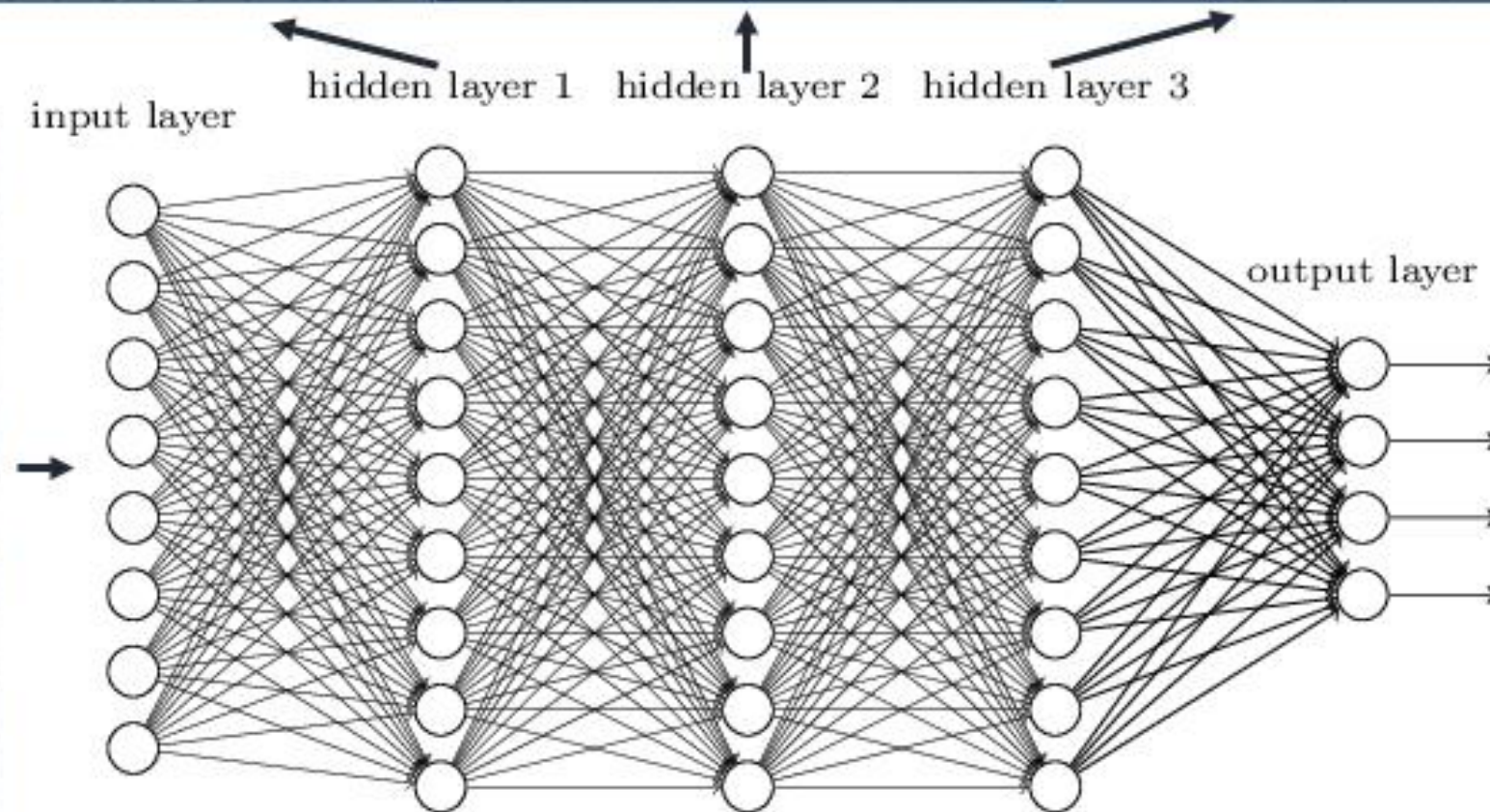
is the partitioning entropy, providing a global validity measure for the clustering results.

- Finally, the dataset T is partitioned into c_{best} subsets using the simple rule : the i th training data point $\mathbf{x}_i$ is assigned to the k th cluster if its membership degree to the k th cluster is the highest, namely

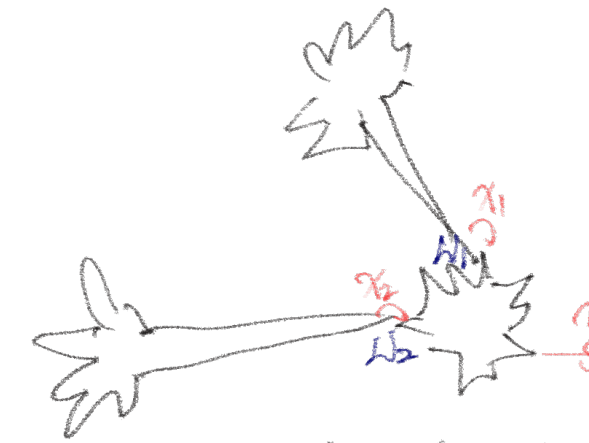
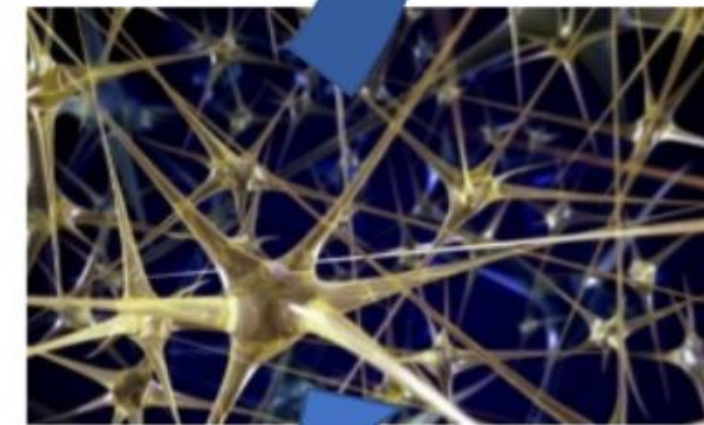
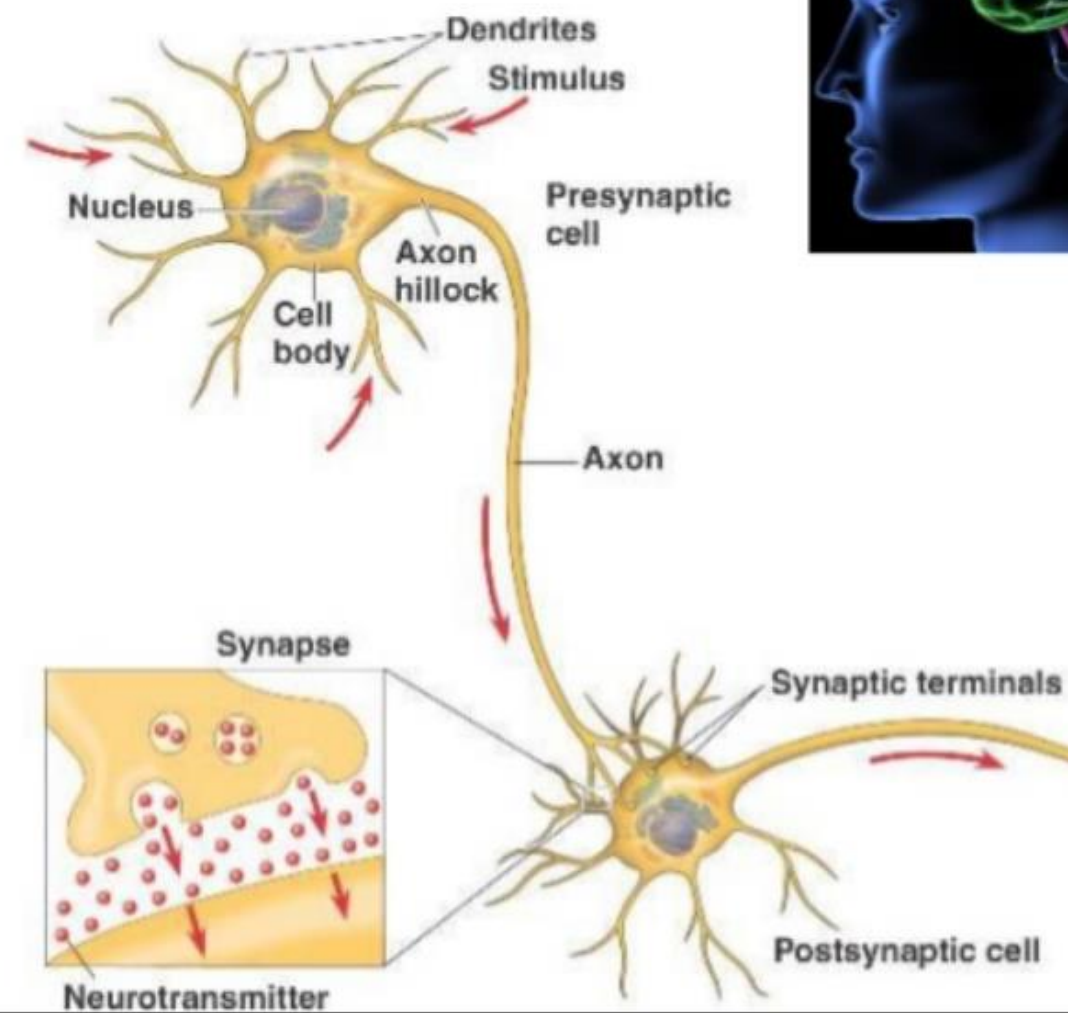
$$k = \arg \max_{j=1}^{c_{best}} u_{ji}$$

Introduction to Neural Network (DNN) 神經網路

Deep neural
networks learn
hierarchical feature
representations



Human Brains



$$\Rightarrow w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

$$y = f(x_1, x_2, x_3, \dots, x_n) \\ = f\left(\sum_{i=1}^n w_i x_i + b\right) = f(z)$$

非線性
轉換

Activation 激活

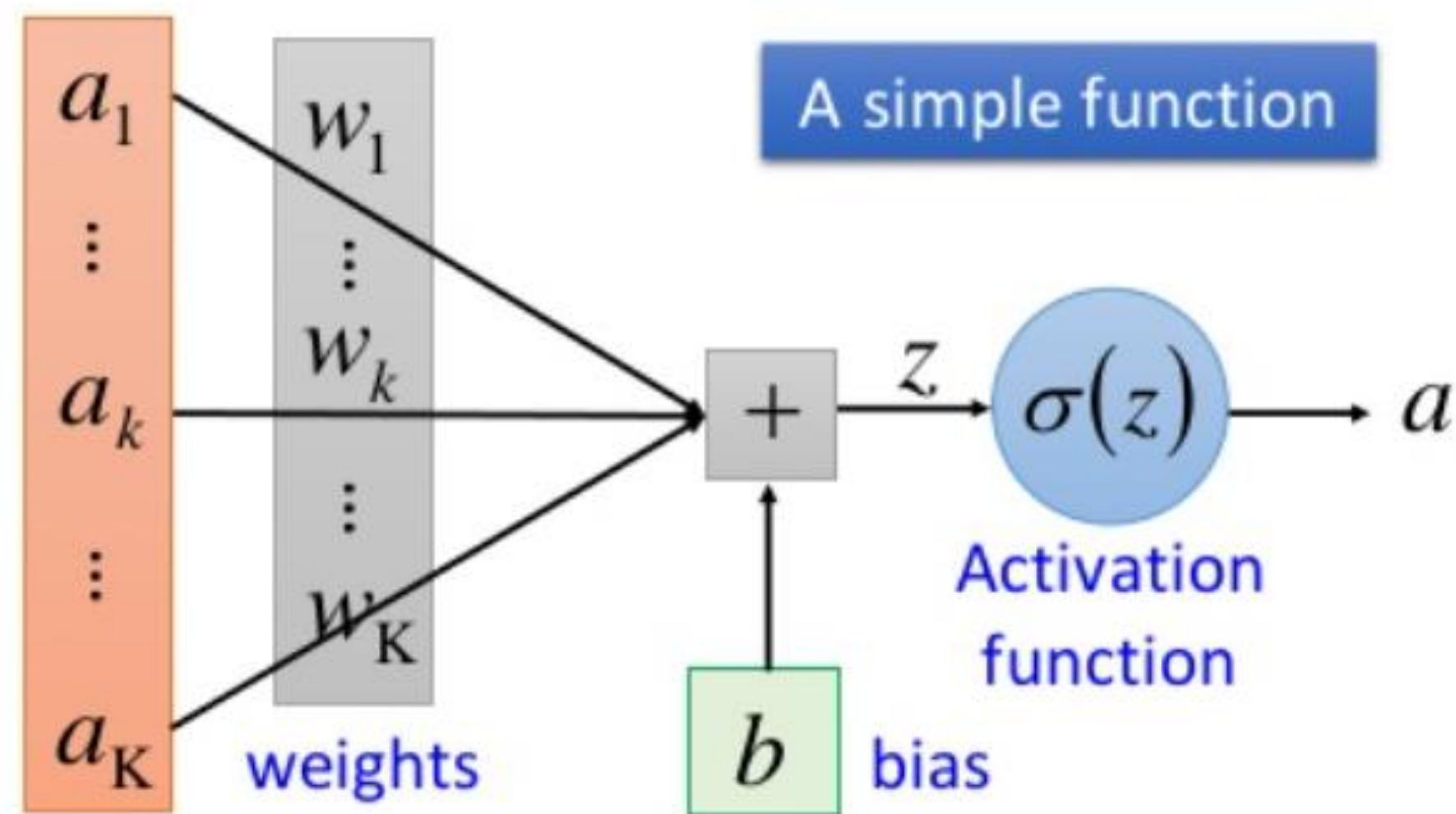
$\Rightarrow$ nonlinear transmit

Neural Network

Neuron

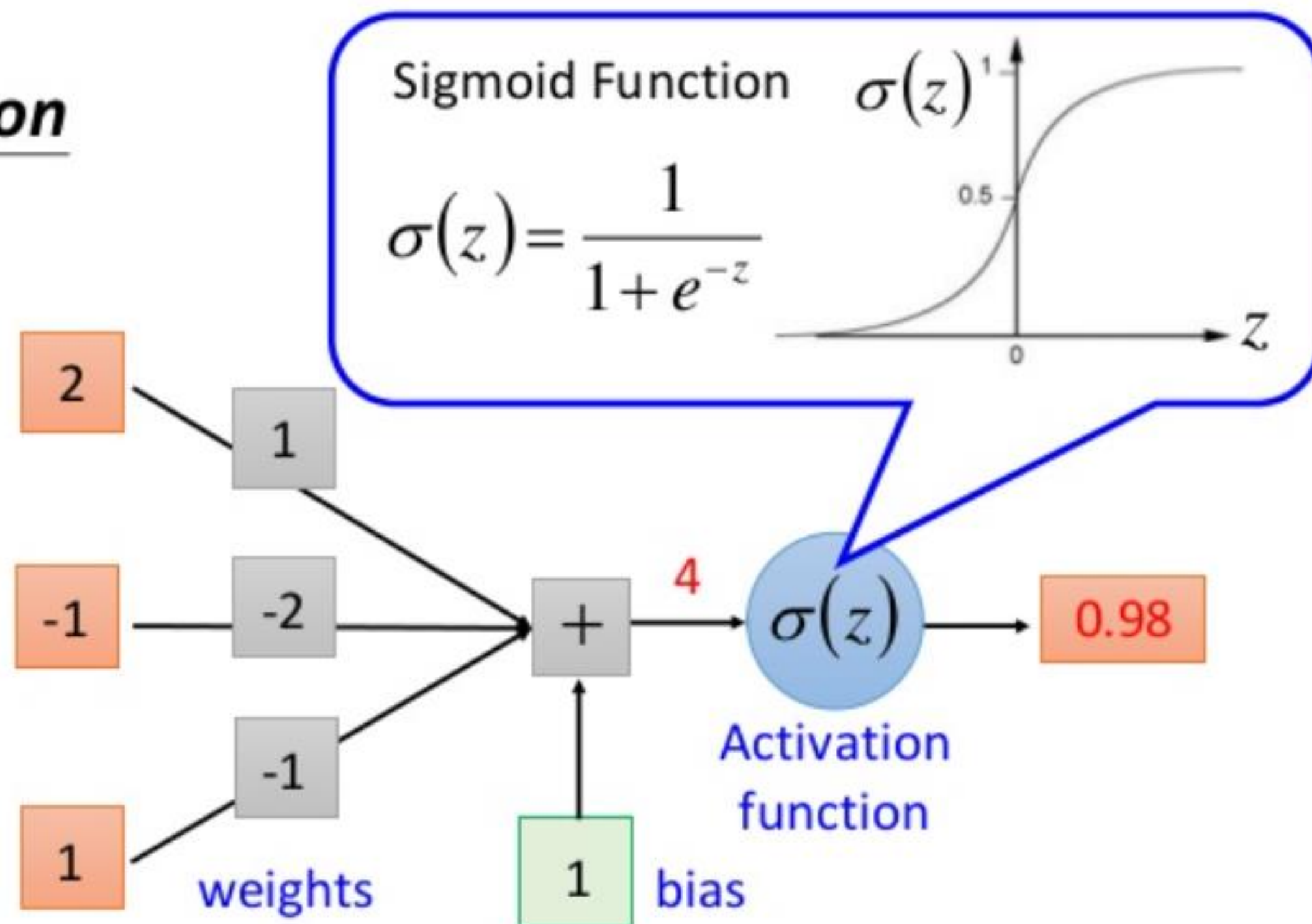
單顆神經元

$$z = a_1 w_1 + \cdots + a_k w_k + \cdots + a_K w_K + b$$



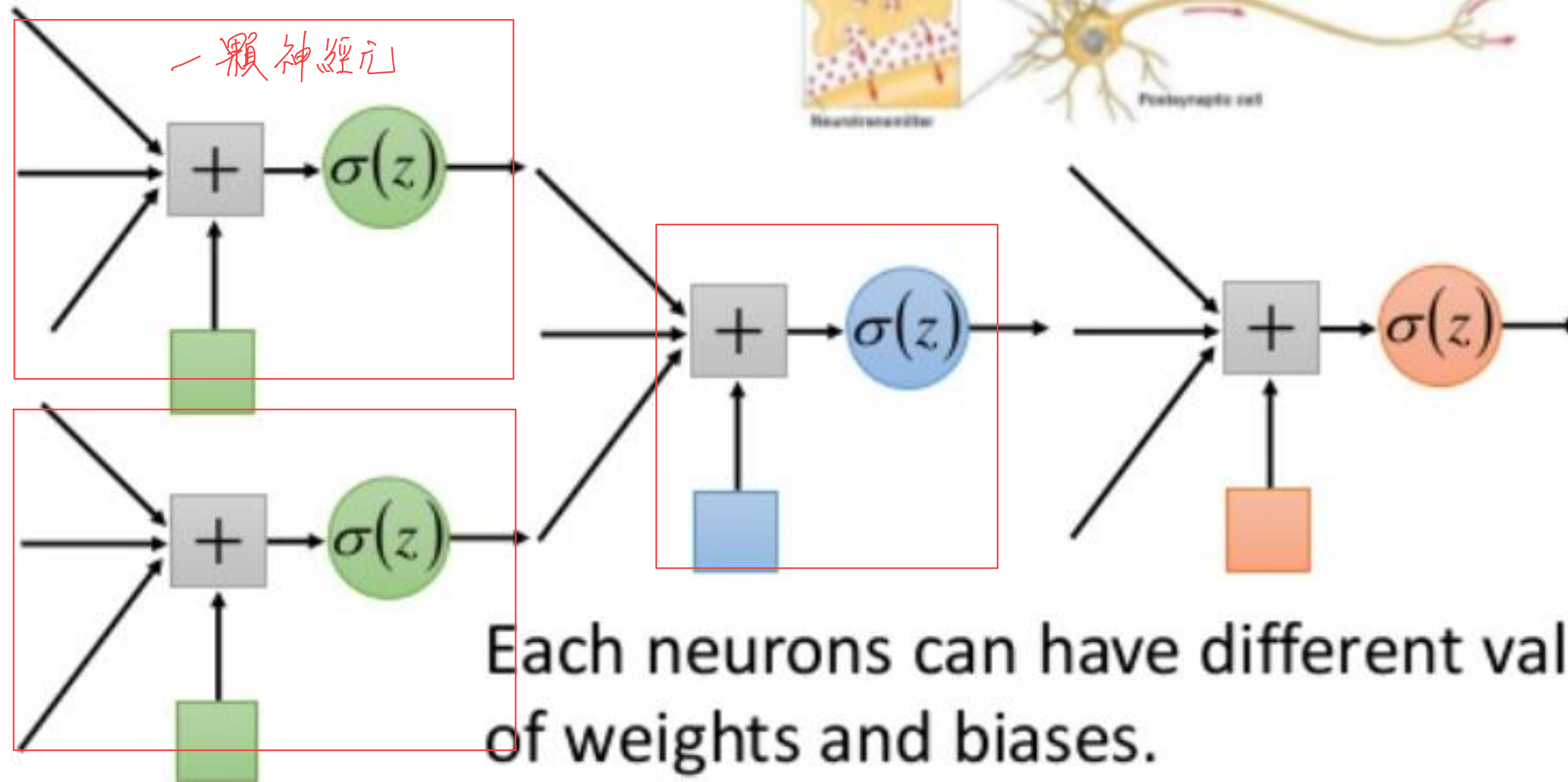
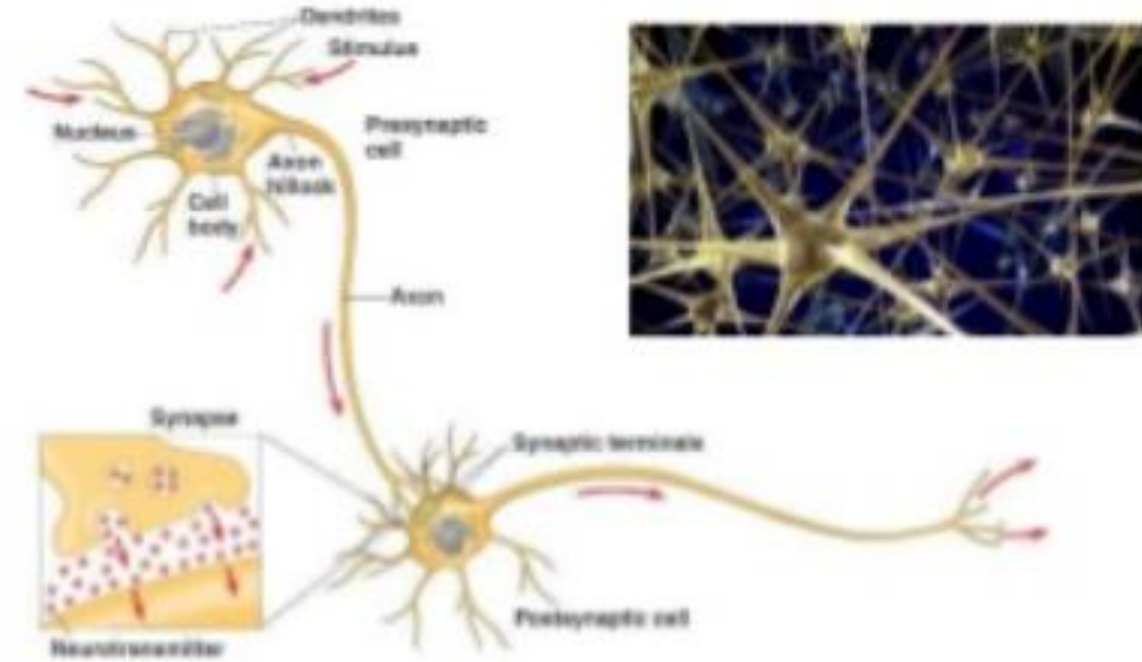
Neural Network

Neuron



Neural Network

Different connections leads to different network structure

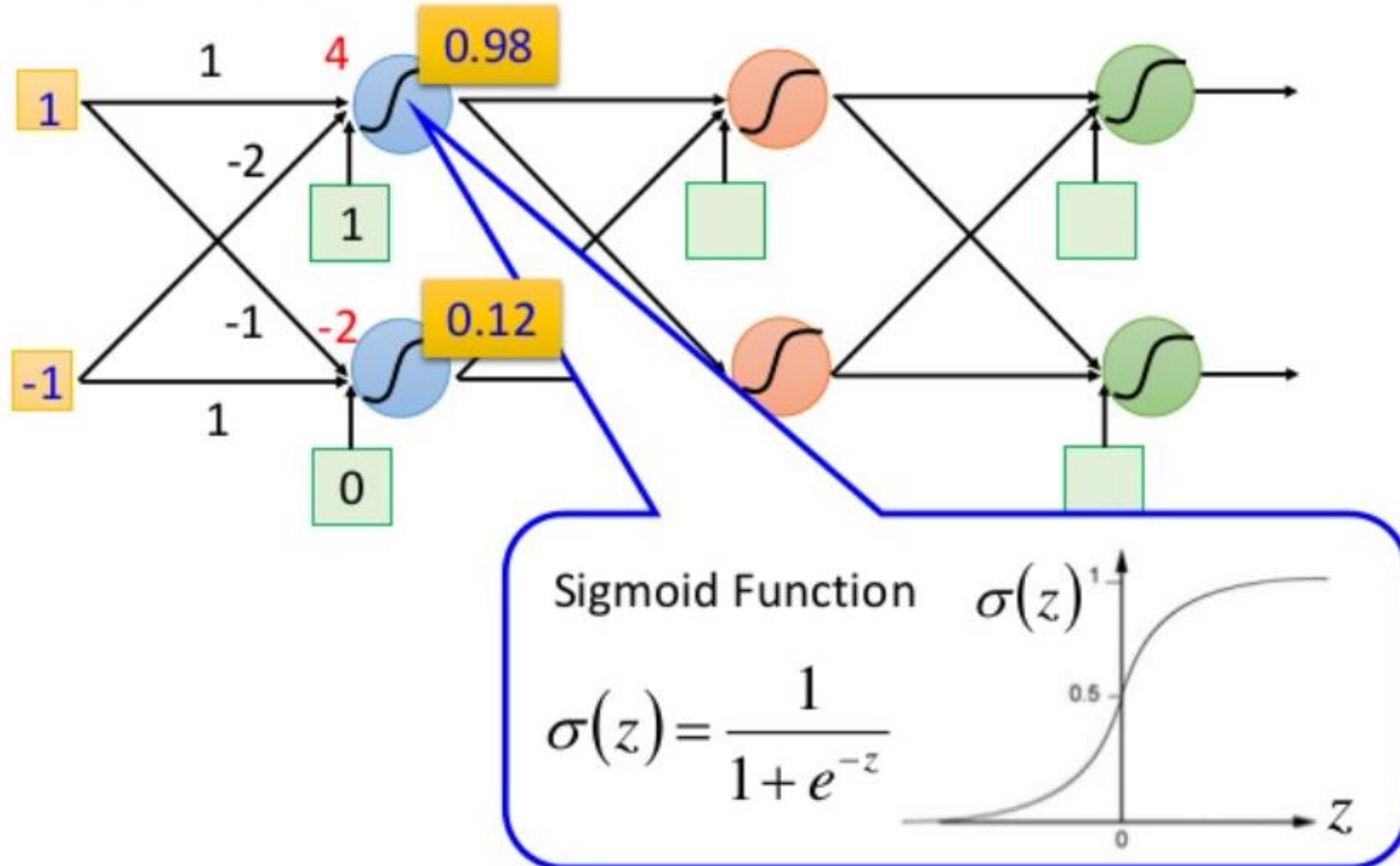


Weights and biases are network parameters θ

這個集合收集
了所有超參數 w, b

超參數集

Fully Connect Feedforward Network



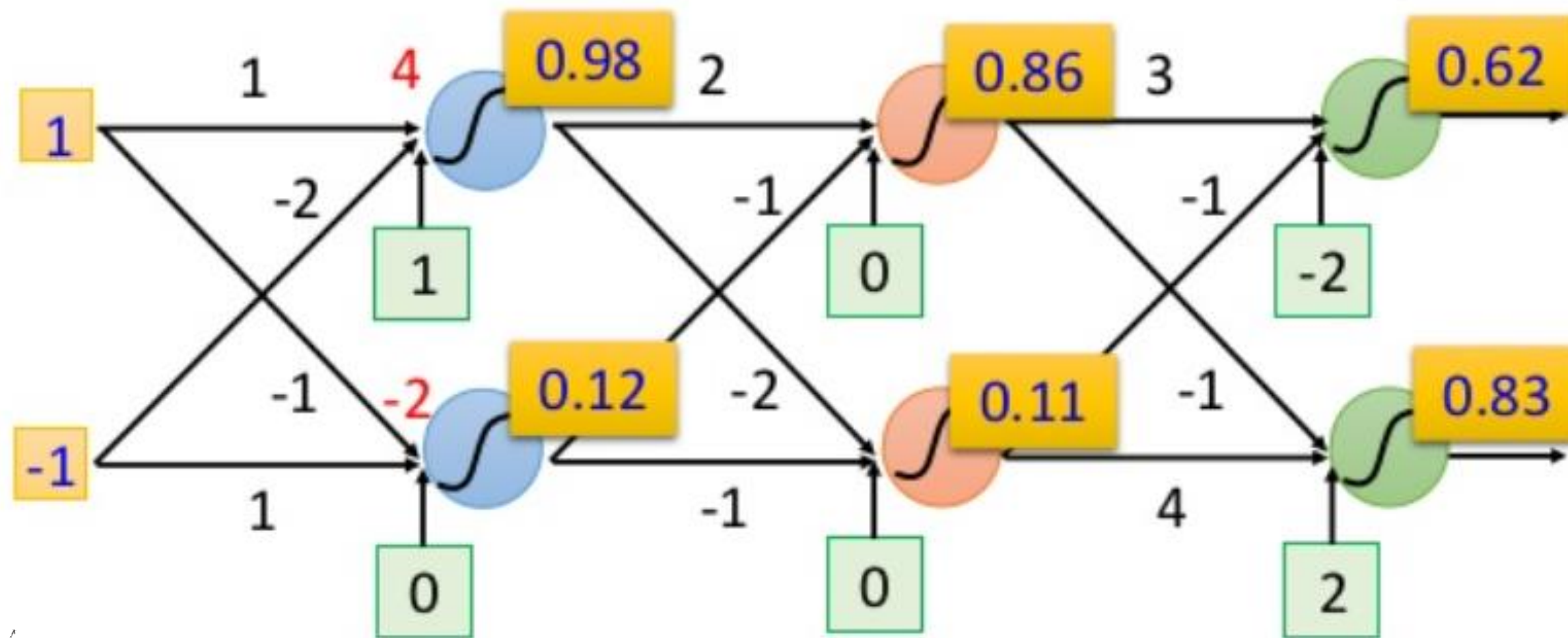
Multilayer perceptron MLP 多層感知機 = Neural Network 神經網路
= artificial neural network (ANN)

超過3層 (> 4)

$\Rightarrow$ DNN (deep neural network)

Fully Connect Feedforward Network

卷積神經網路 Convolution neural network
 $\Rightarrow$ CNN = 卷積層串接 MLP



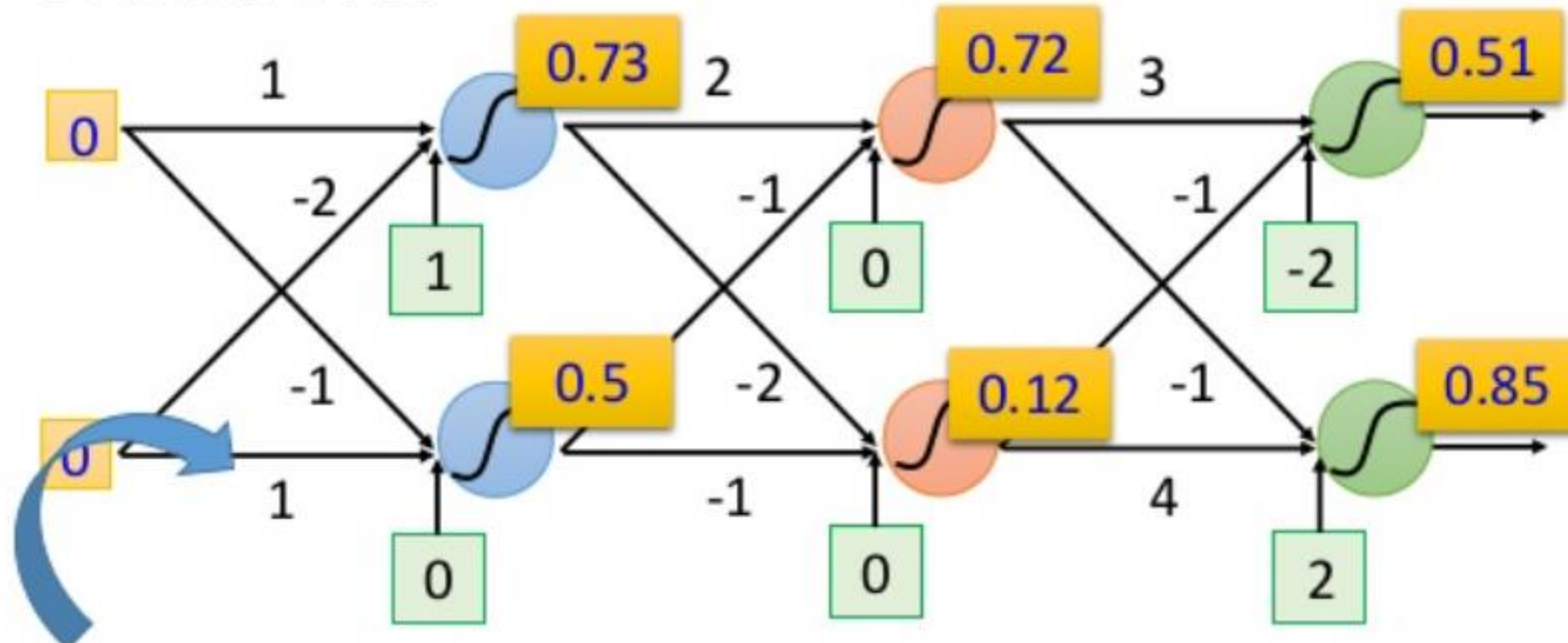
I/p layer

2. hidden layer

O/p layer

4 layer fully-connected feedforward neural networks

Fully Connect Feedforward Network



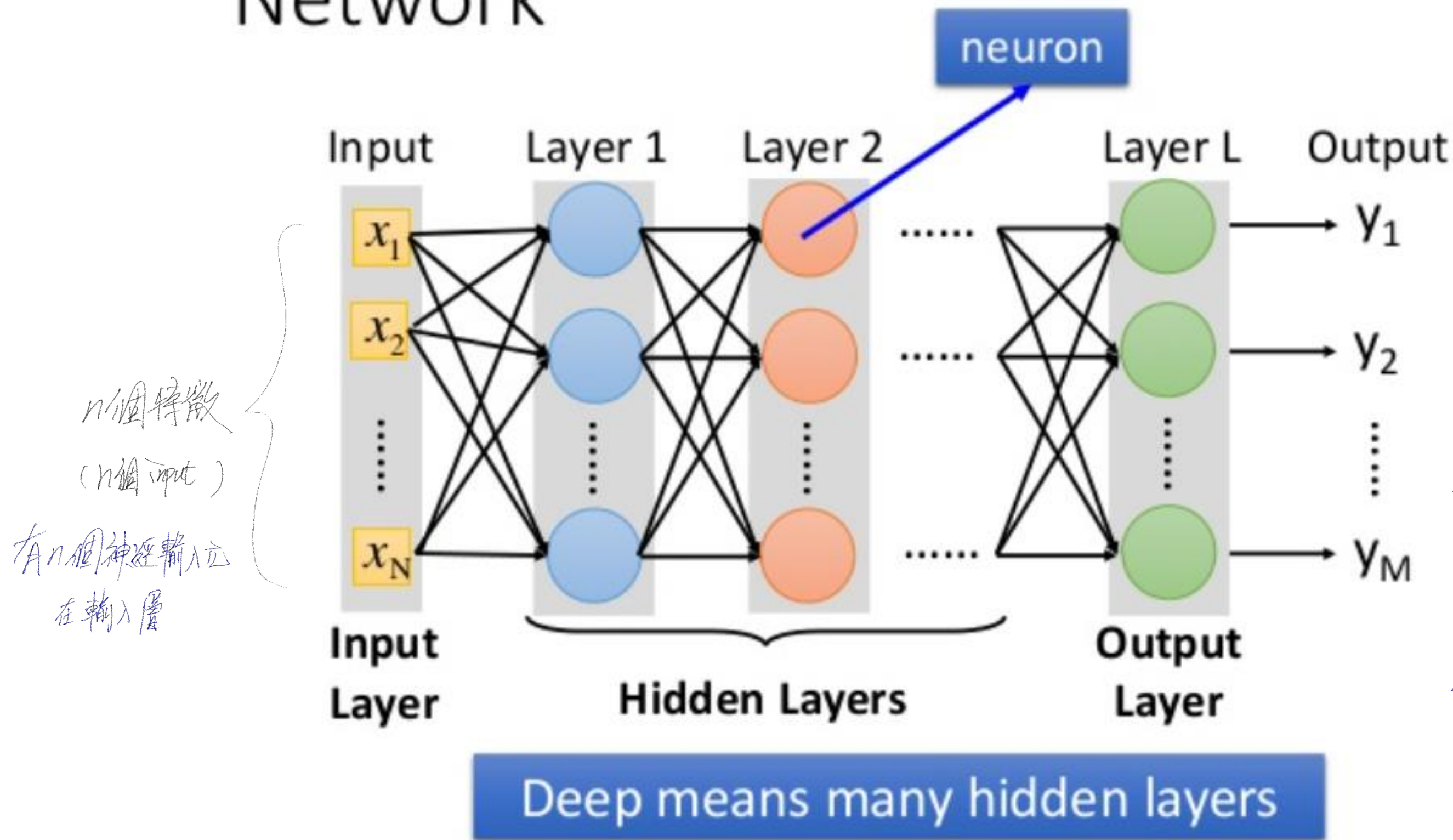
This is a function.

Input vector, output vector $f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$ $f\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0.51 \\ 0.85 \end{bmatrix}$

Given parameters θ , define a function

Given network structure, define a function set

Fully Connect Feedforward Network



Ex. Iris data set.

1. # input nodes
(4个) 特征数量

2. # output nodes
采用最短路方法

ex. 1st - class data label = [1, 0, 0]
 training { 2nd - class data label = [0, 1, 0]
 3rd - class data label = [0, 0, 1]

3. # hidden layer = user define (参数)

4. # neurons in each hidden layer

⇒ user define. (参数)

Output Layer (Option)

- Softmax layer as the output layer

Ordinary Layer

$$z_1 \rightarrow \sigma \rightarrow y_1 = \sigma(z_1)$$

$$z_2 \rightarrow \sigma \rightarrow y_2 = \sigma(z_2)$$

$$z_3 \rightarrow \sigma \rightarrow y_3 = \sigma(z_3)$$

In general, the output of network can be any value.

May not be easy to interpret

$y_1 = 0.23$
 $y_2 = 0.87 \rightarrow$ 响应最大 $\rightarrow$ 判决为
 $y_3 = 0.67$ response max the data is class 2

Output Layer (Option)

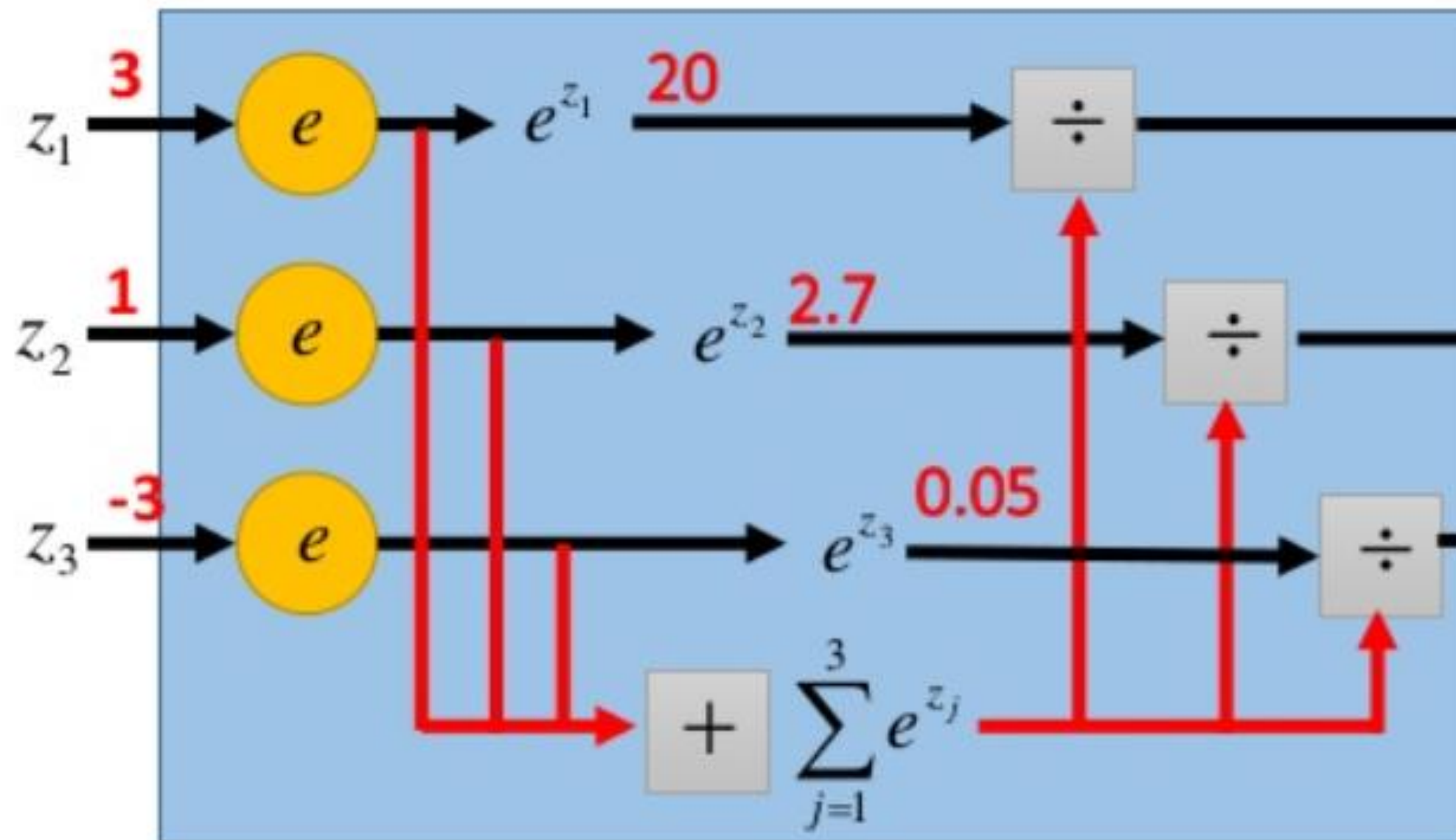
- Softmax layer as the output layer

Probability:

■ $1 > y_i > 0$

■ $\sum_i y_i = 1$

Softmax Layer



→ 屬於 class 1 的 prob.

$$y_1 = e^{z_1} / \sum_{j=1}^3 e^{z_j}$$

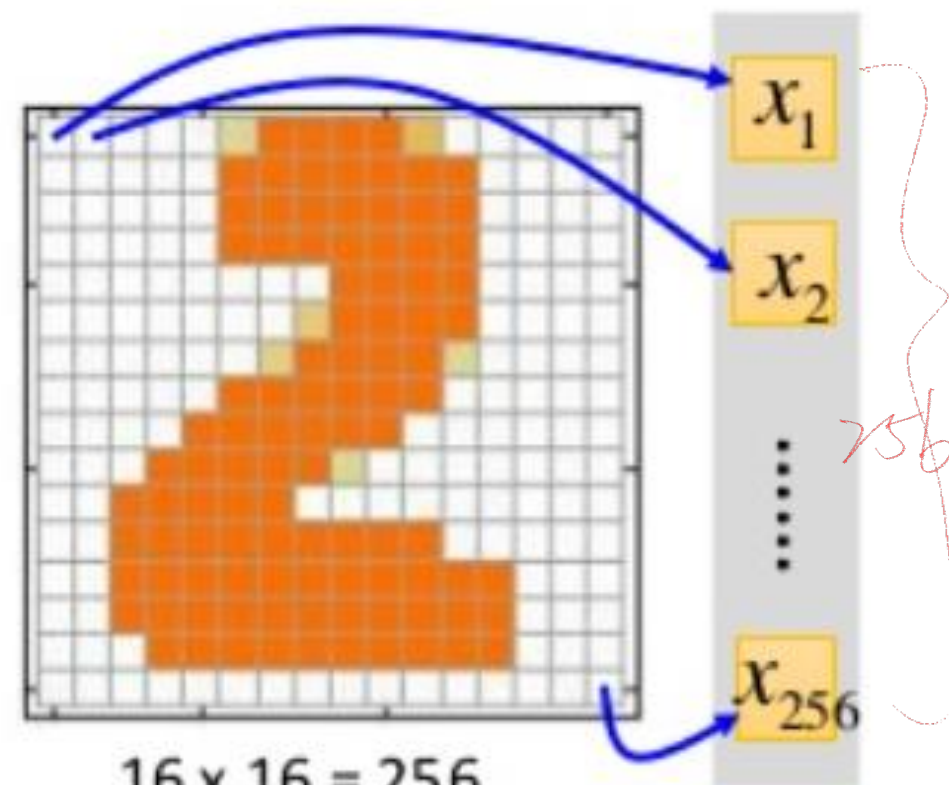
$$y_2 = e^{z_2} / \sum_{j=1}^3 e^{z_j} \Rightarrow \text{屬於 class 2 的 prob.}$$

$$y_3 = e^{z_3} / \sum_{j=1}^3 e^{z_j} \Rightarrow \therefore \text{class 3 = prob.}$$

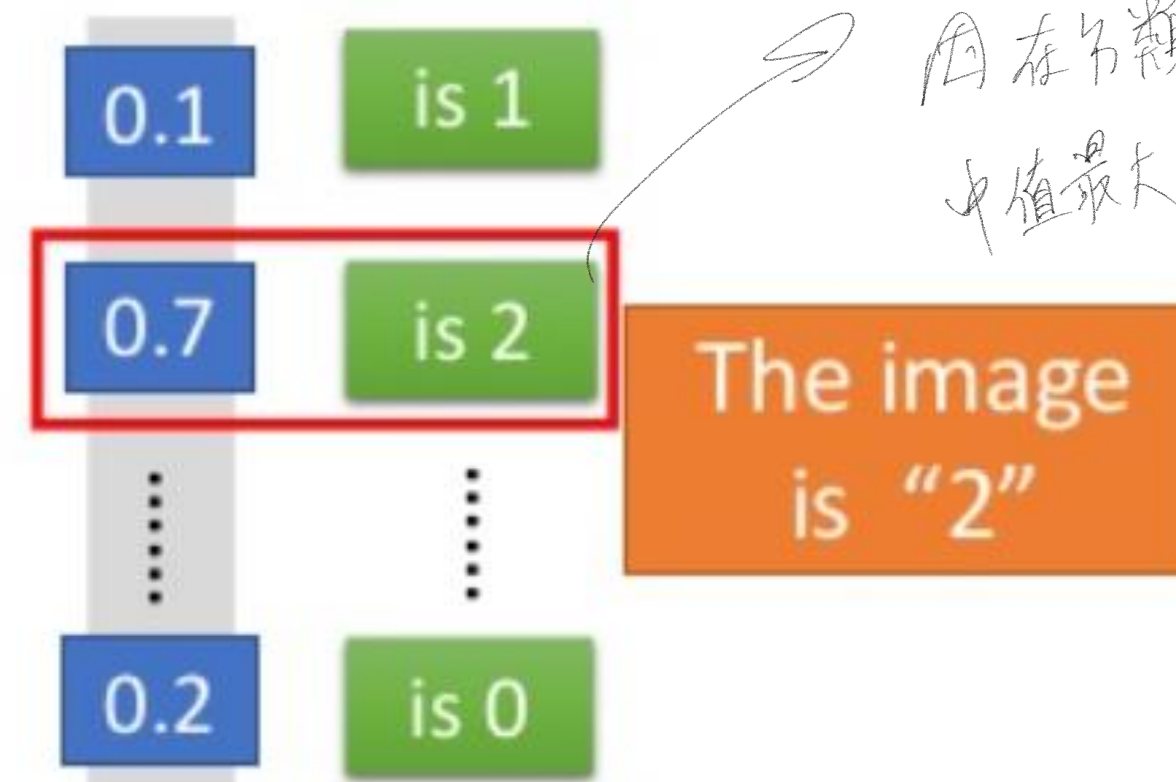
Example Application



Input



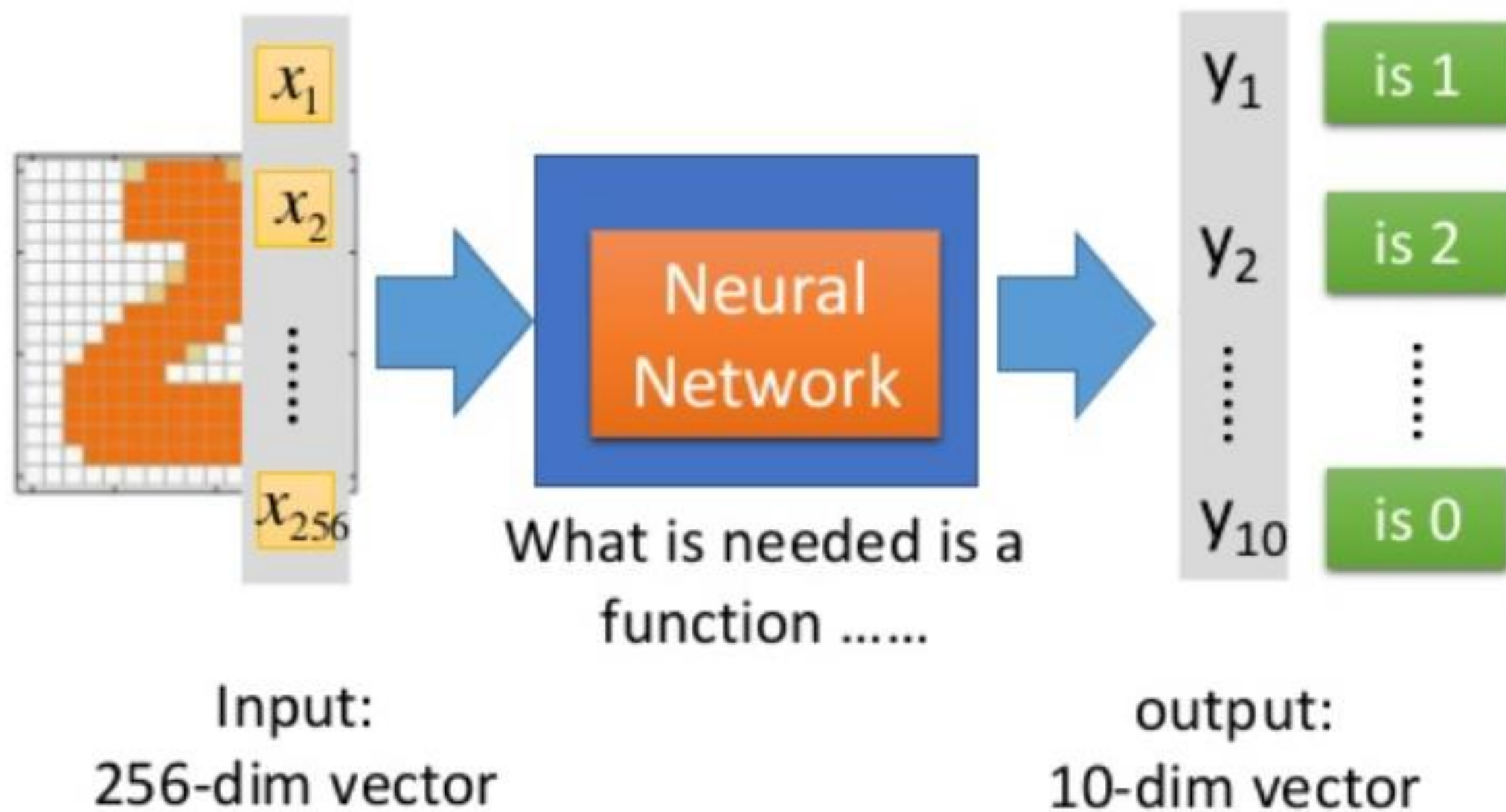
Output



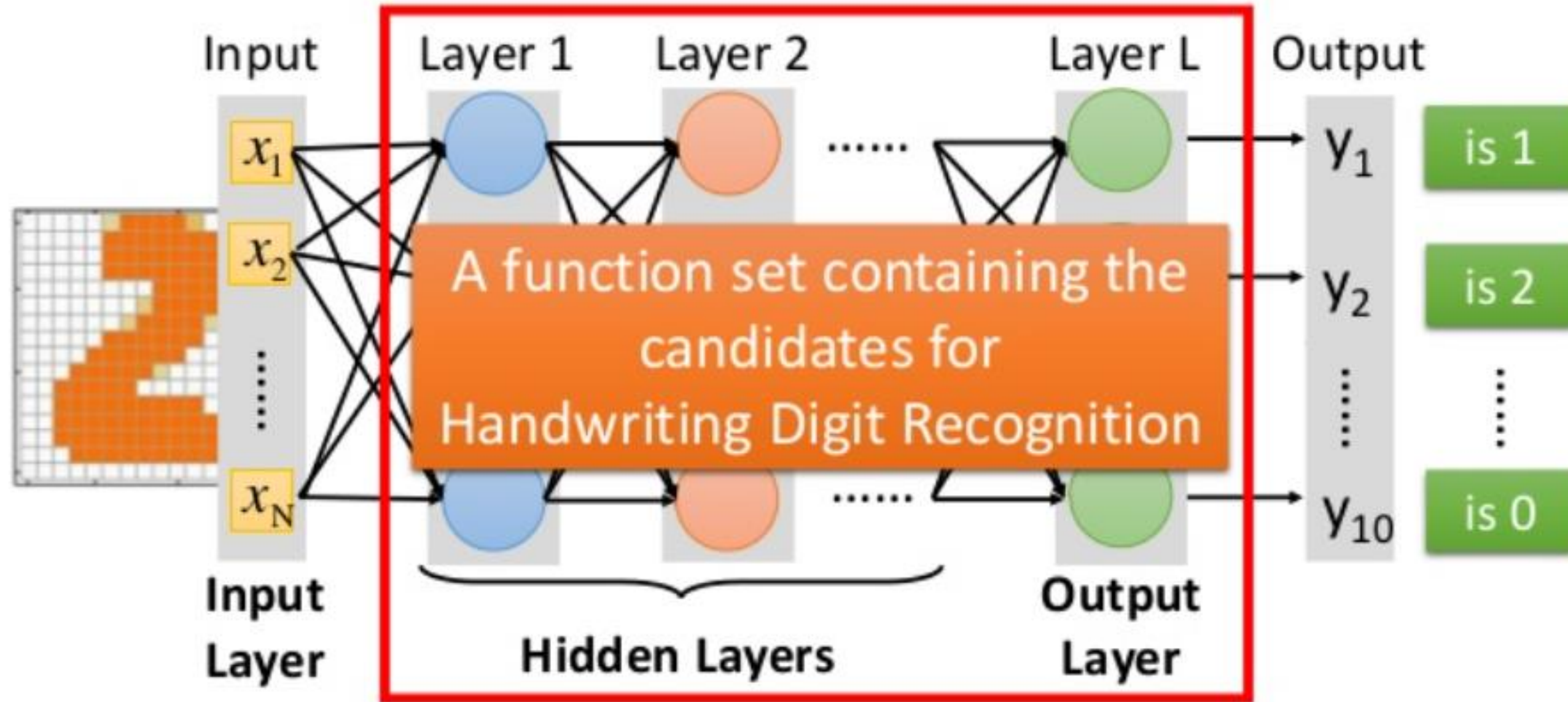
Each dimension represents the confidence of a digit.

Example Application

- Handwriting Digit Recognition

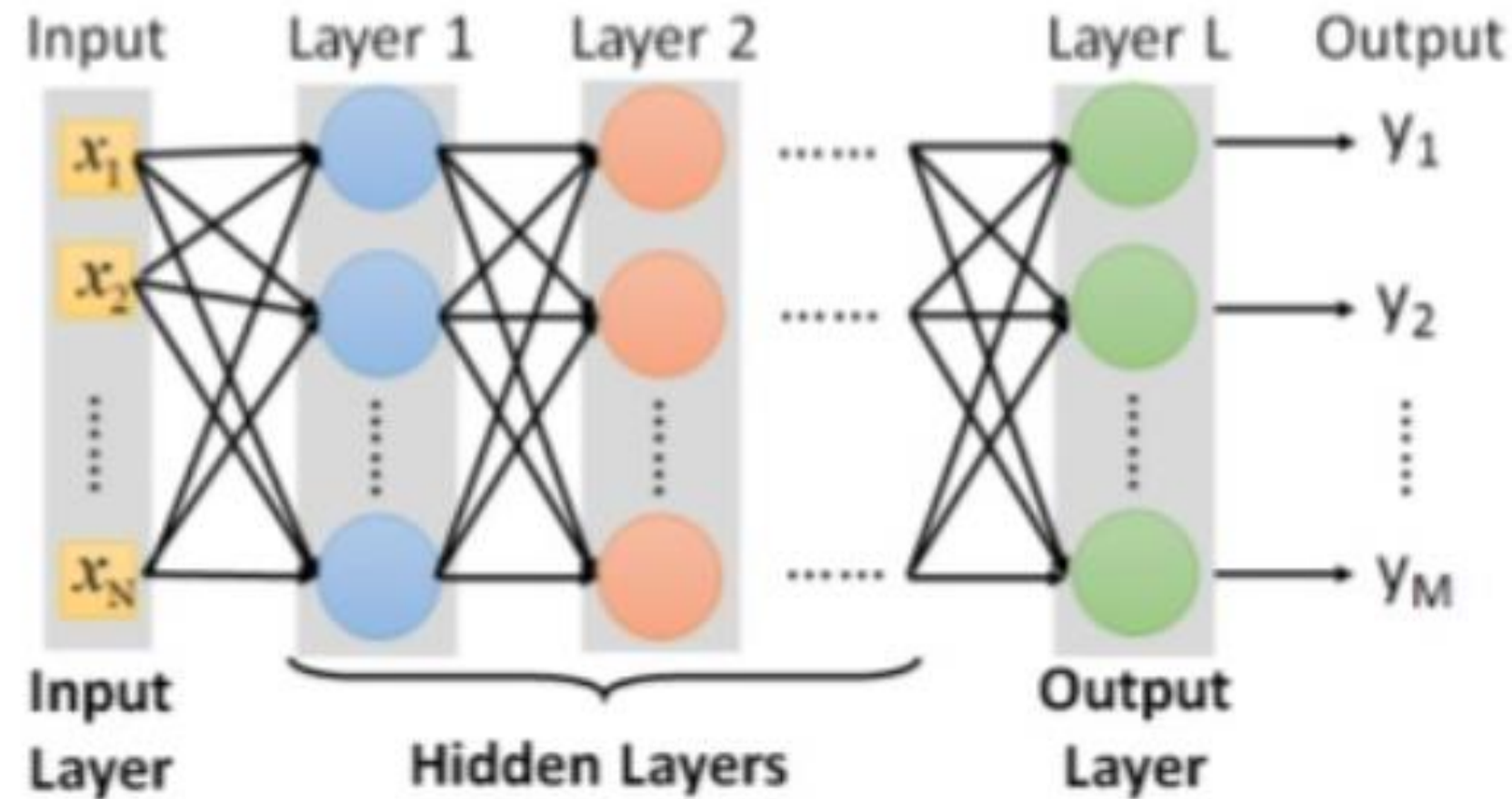


Example Application



You need to decide the network structure to let a good function in your function set.

FAQ



- Q: How many layers? How many neurons for each layer?

Trial and Error

+

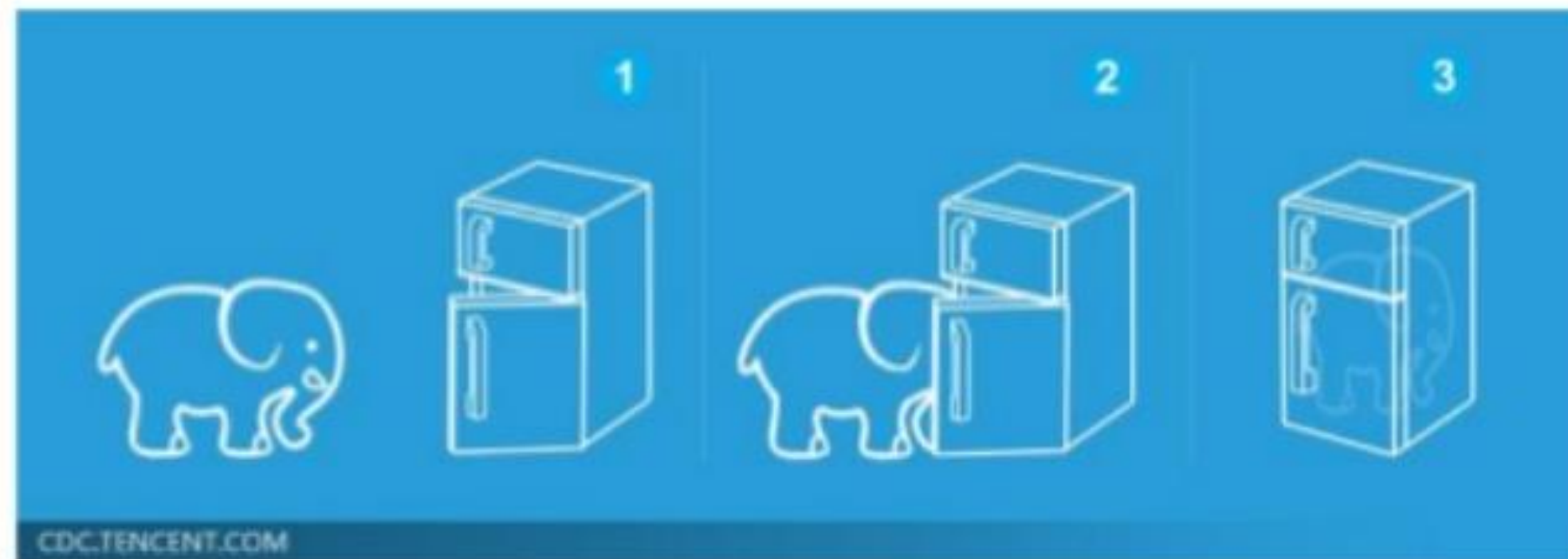
Intuition

- Q: Can the structure be automatically determined?

Three Steps for Deep Learning

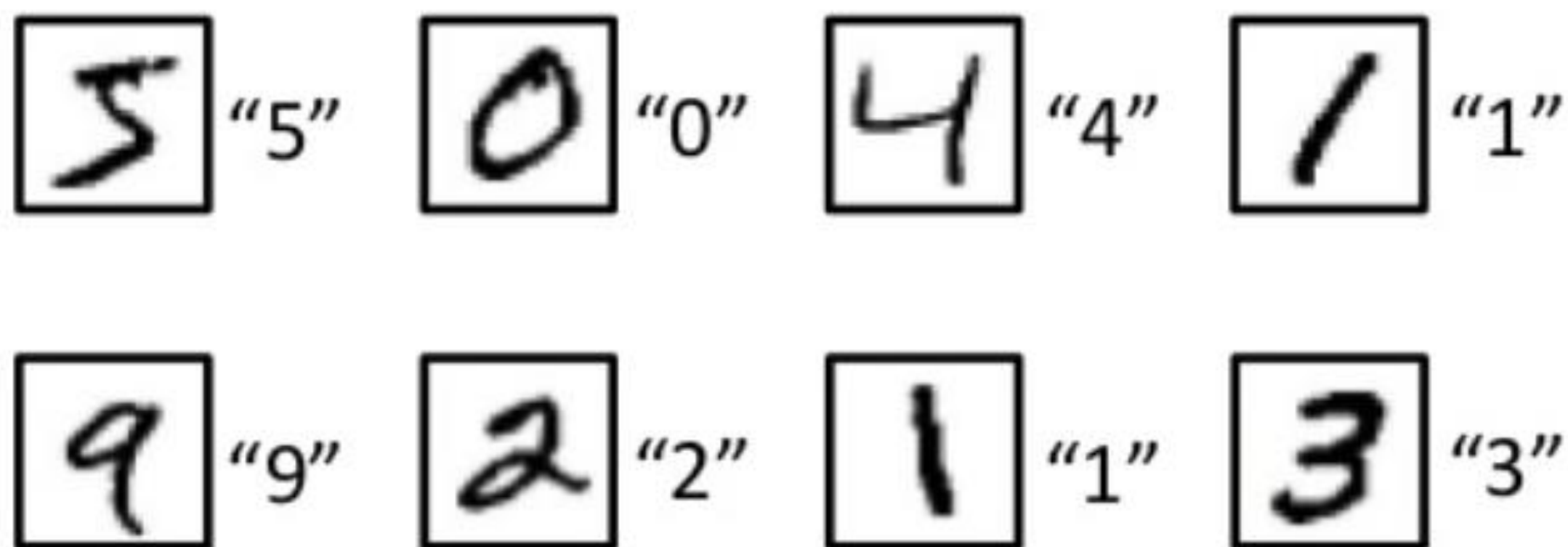


Deep Learning is so simple



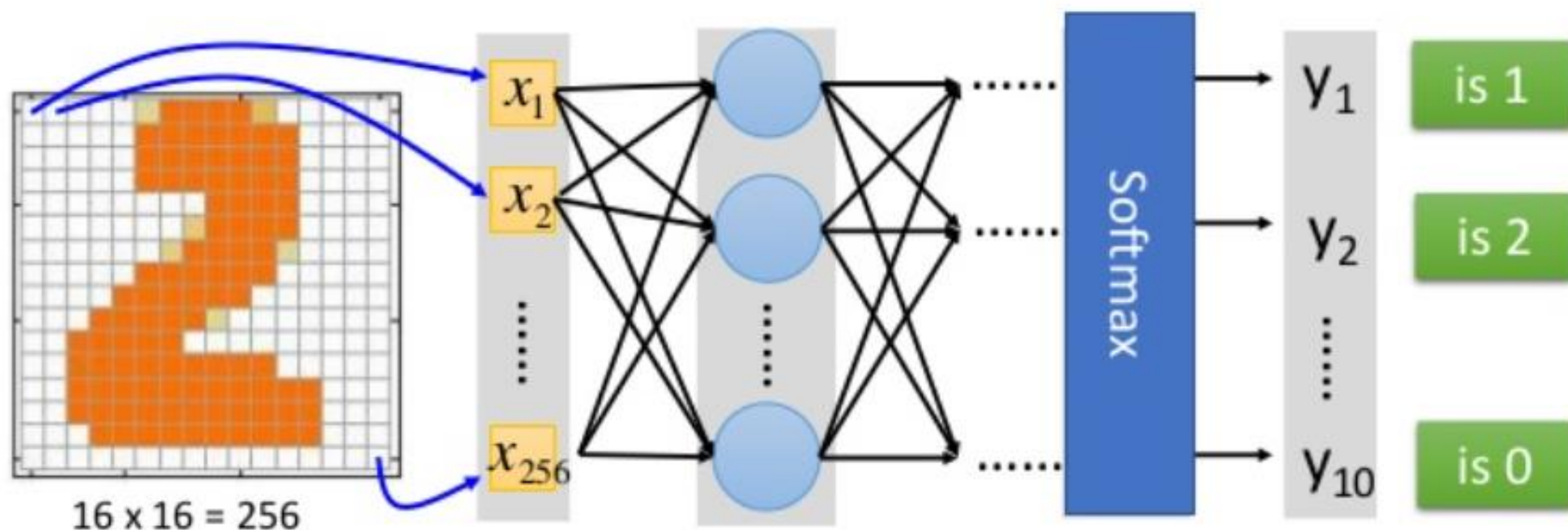
Training Data

- Preparing training data: images and their labels



The learning target is defined on the training data.

Learning Target



16 x 16 = 256

Ink $\rightarrow$ 1

No ink $\rightarrow$ 0

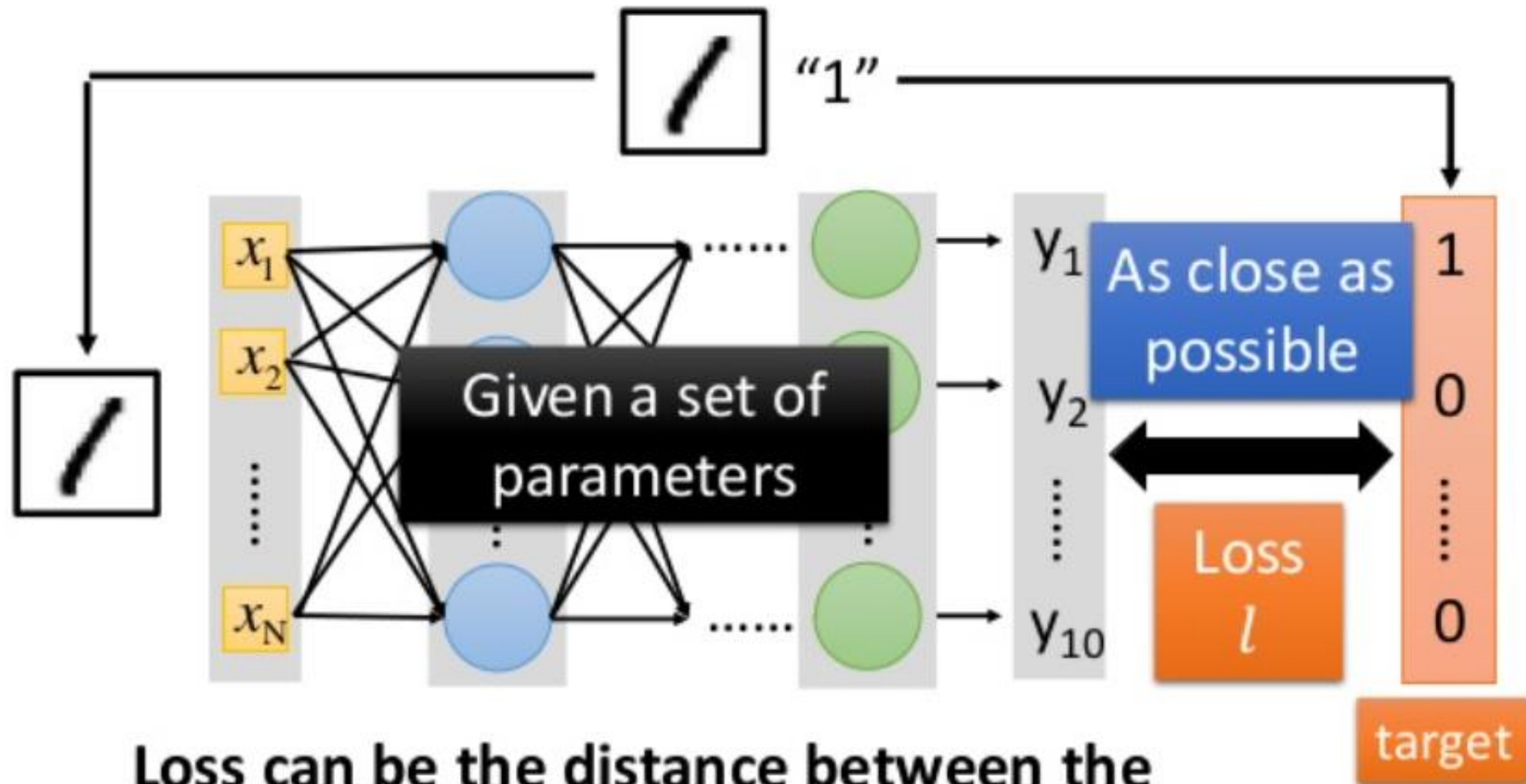
The learning target is

Input:  $\rightarrow$ y_1 has the maximum value

Input:  $\rightarrow$ y_2 has the maximum value

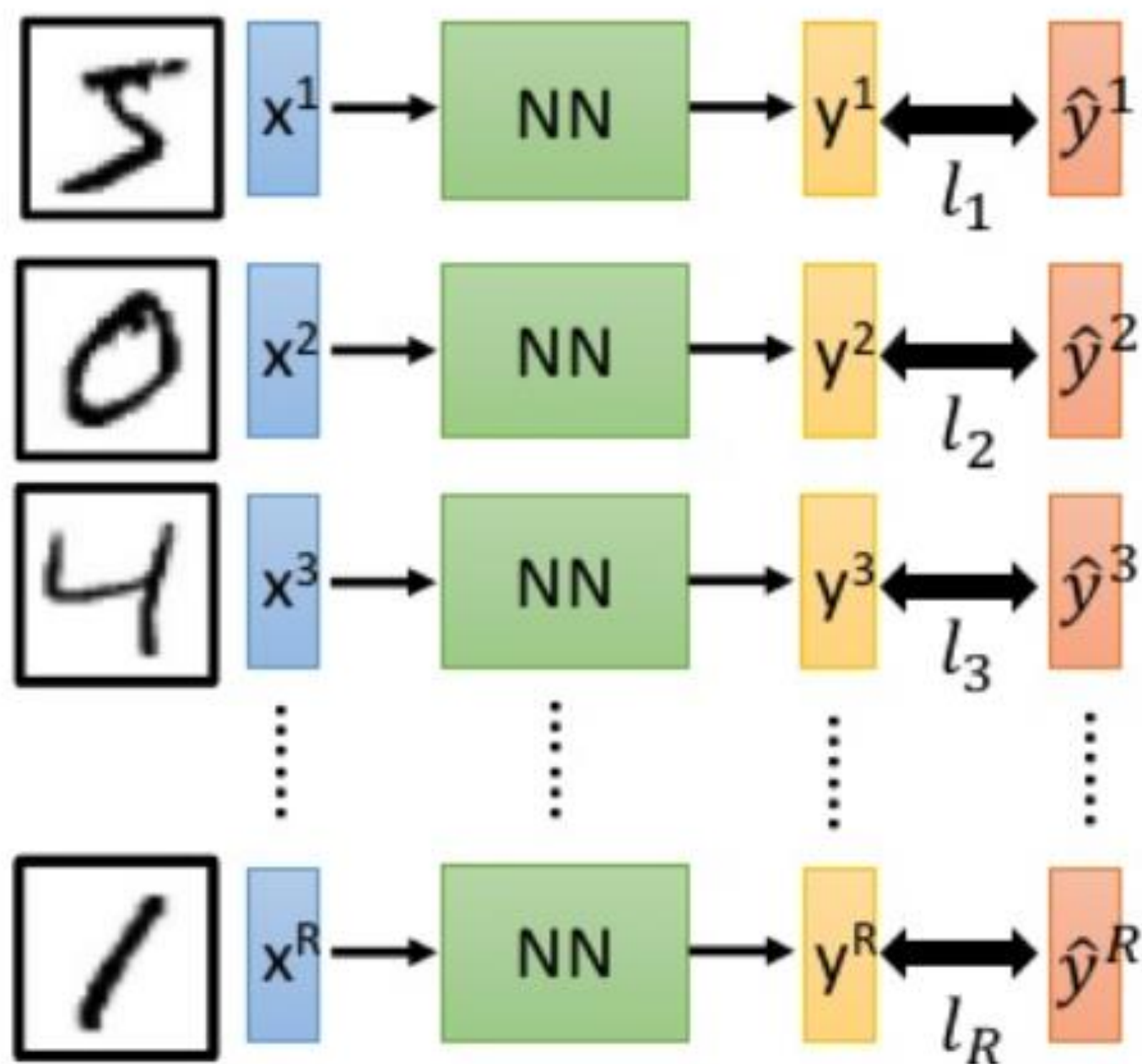
LOSS

A good function should make the loss of all examples as small as possible.



Total Loss

For all training data ...



Total Loss:

$$L = \sum_{r=1}^R l_r$$

As small as possible

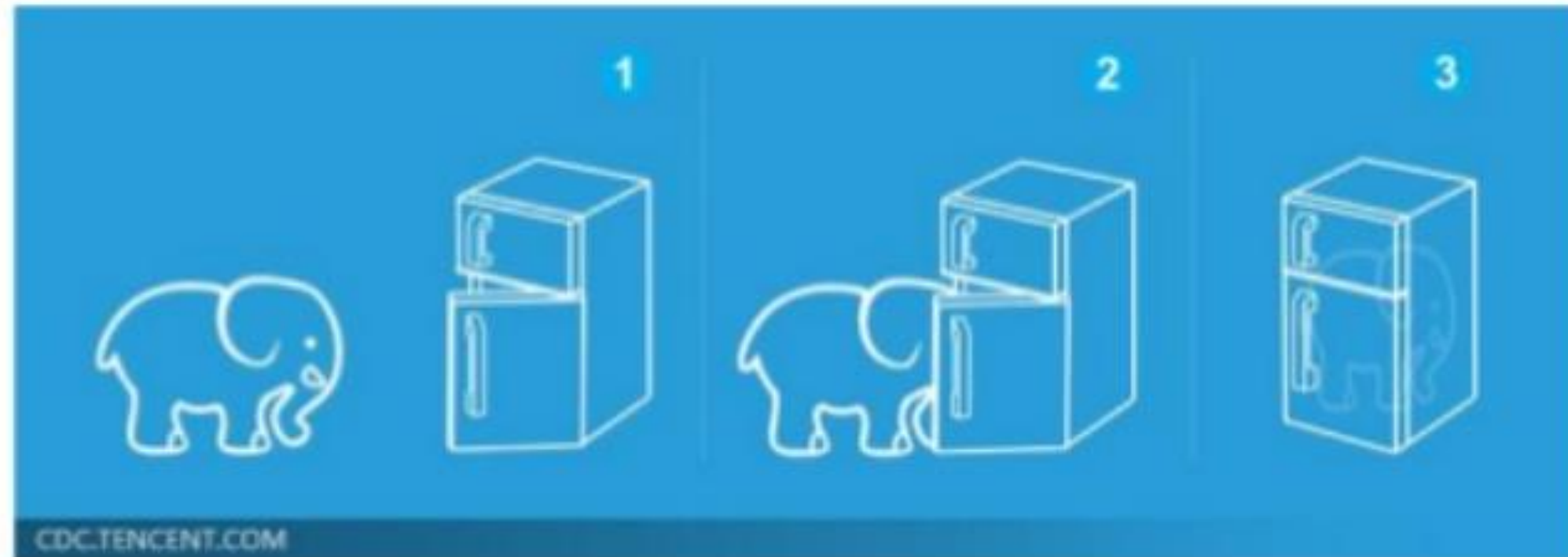
Find *a function in function set* that minimizes total loss L

Find *the network parameters* θ^* that minimize total loss L

Three Steps for Deep Learning



Deep Learning is so simple



How to pick the best function

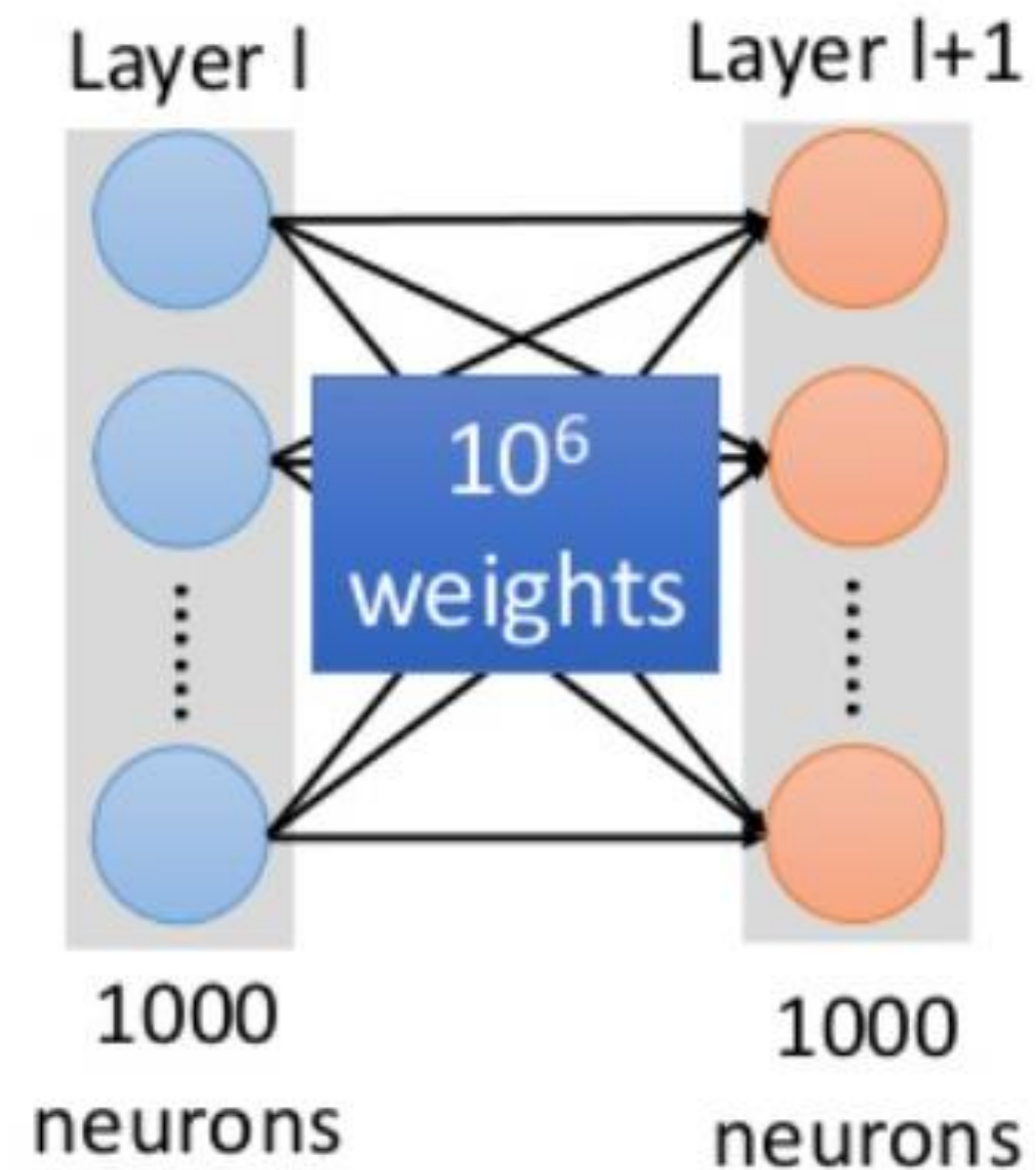
Find network parameters θ^* that minimize total loss L

Enumerate all possible values

Network parameters $\theta =$
 $\{w_1, w_2, w_3, \dots, b_1, b_2, b_3, \dots\}$

Millions of parameters

E.g. speech recognition: 8 layers and
1000 neurons each layer



Gradient Descent

Network parameters $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

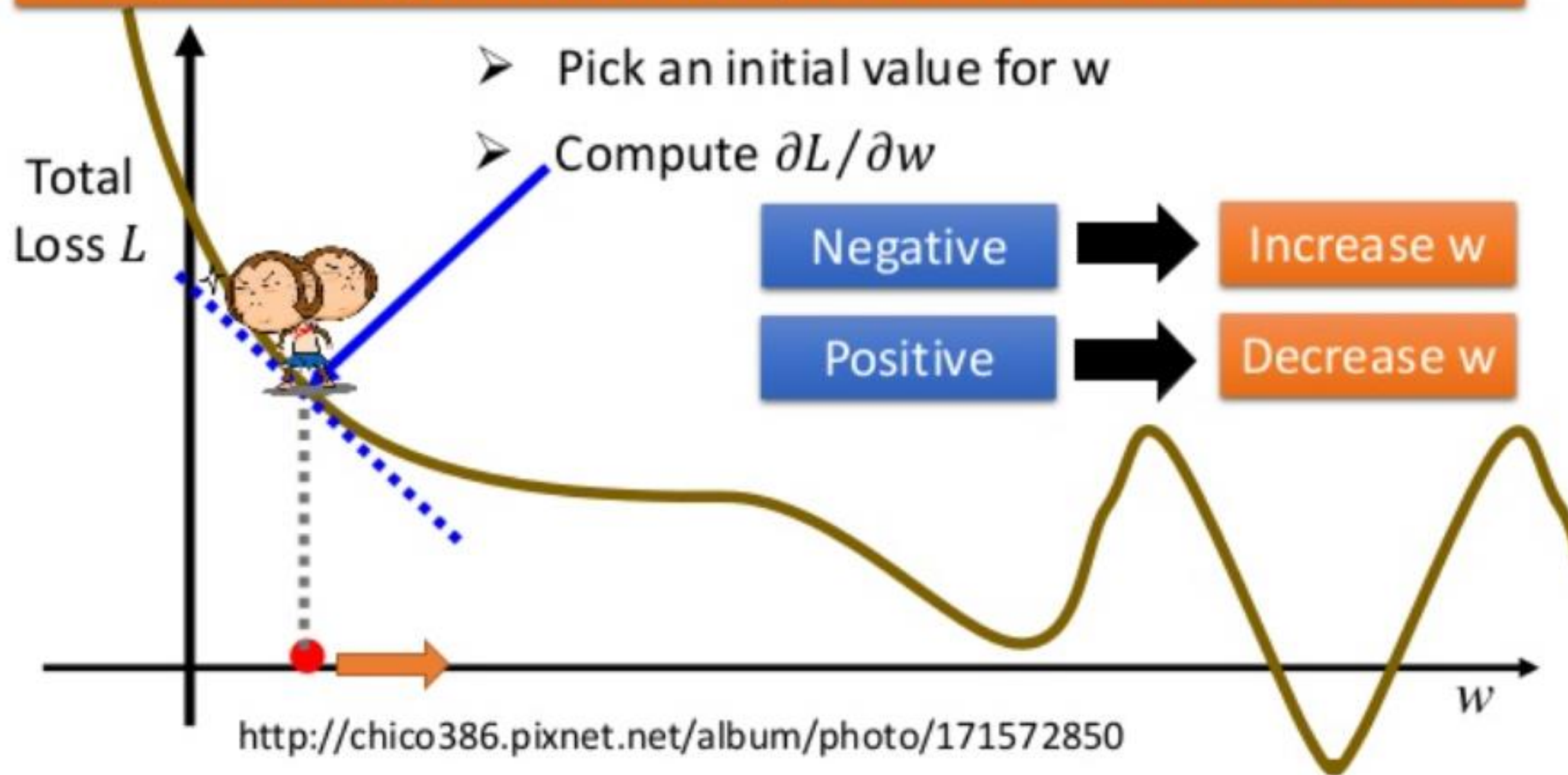
Find network parameters θ^* that minimize total loss L



Gradient Descent

Network parameters $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

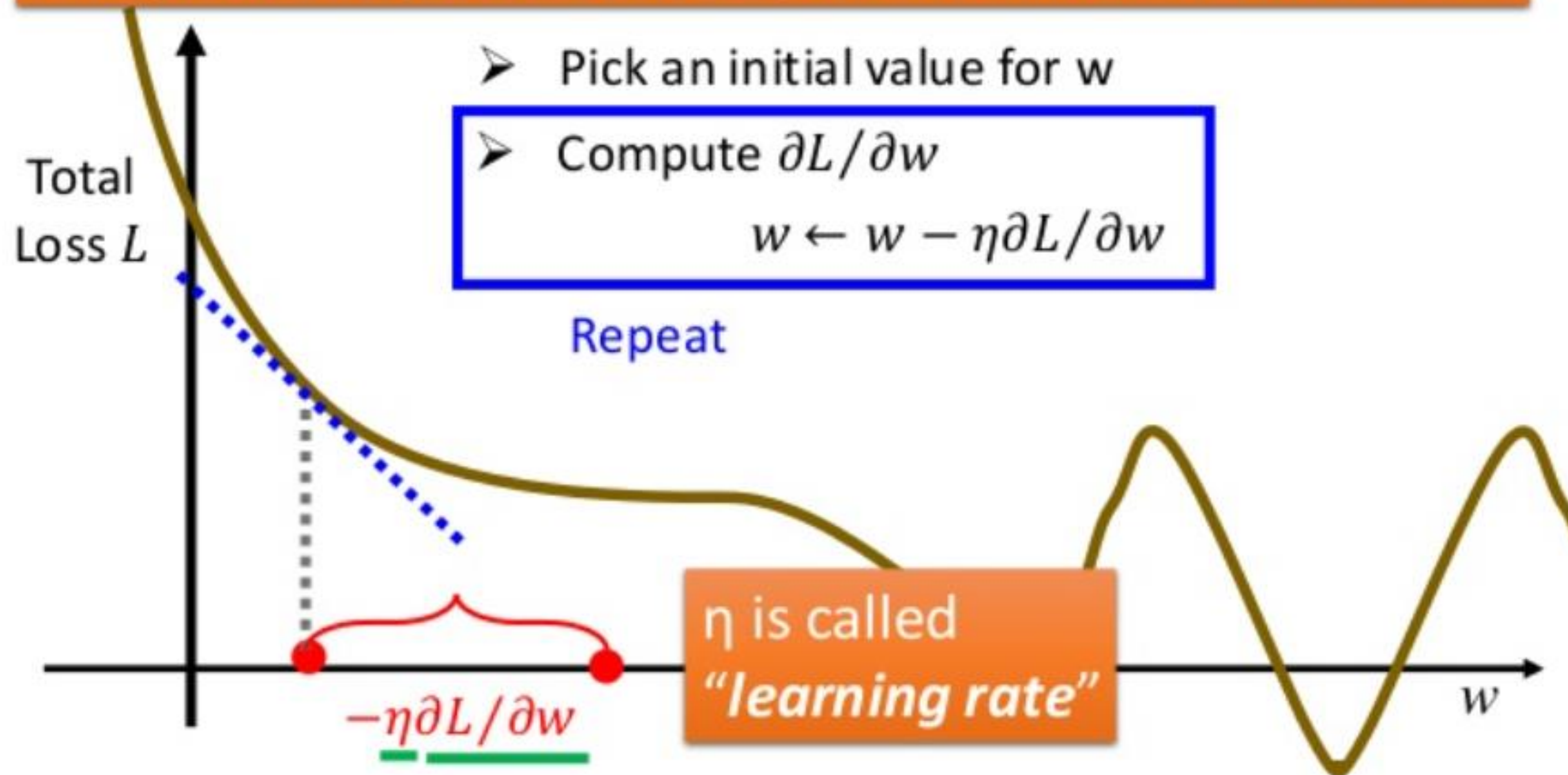
Find network parameters θ^* that minimize total loss L



Gradient Descent

Network parameters $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

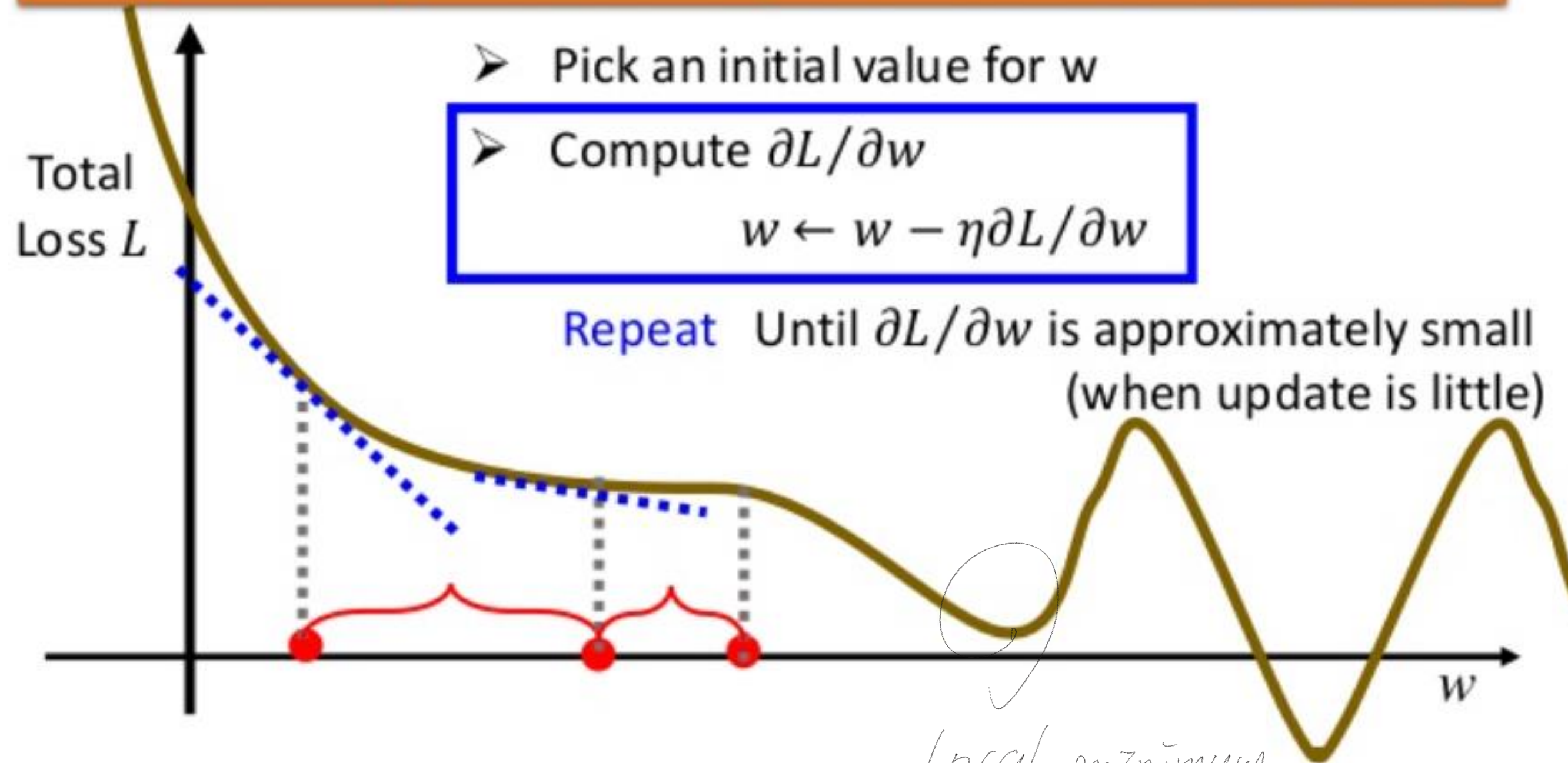
Find network parameters θ^* that minimize total loss L



Gradient Descent

Network parameters $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

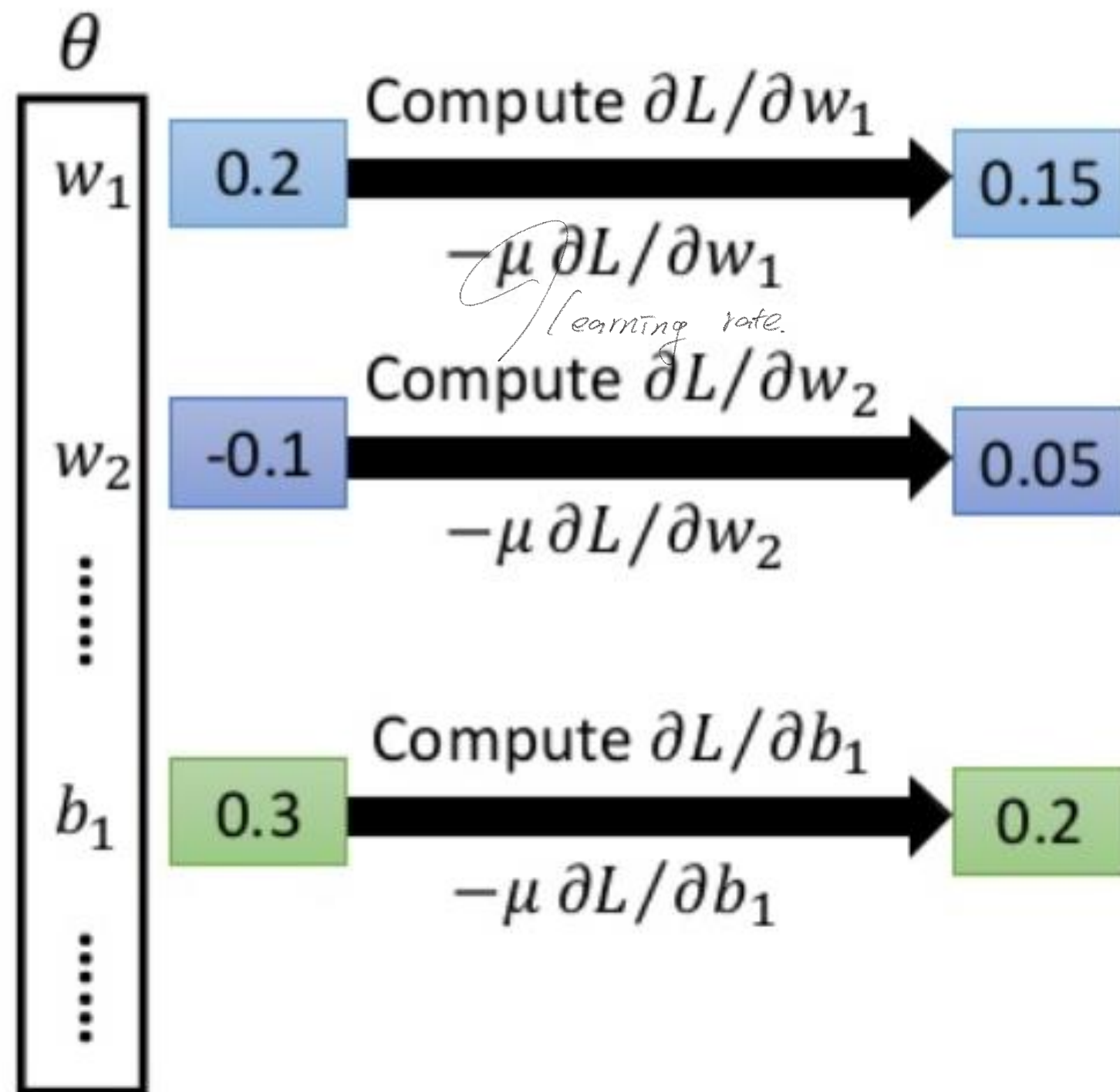
Find network parameters θ^* that minimize total loss L



dis. 無法避免
local 問題

local minimum
並非 global minimum

Gradient Descent

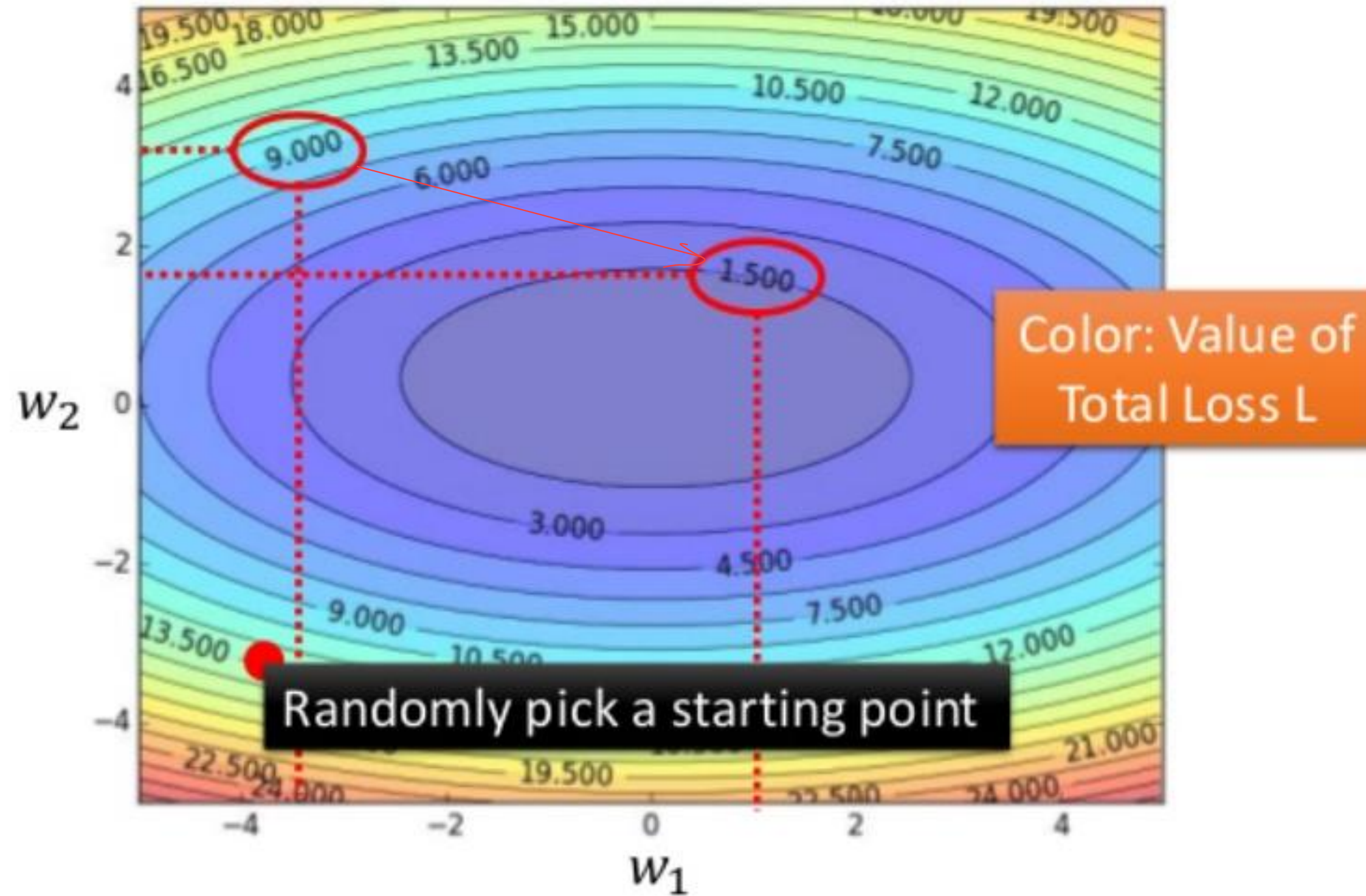


$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_1} \\ \vdots \end{bmatrix}$$

gradient

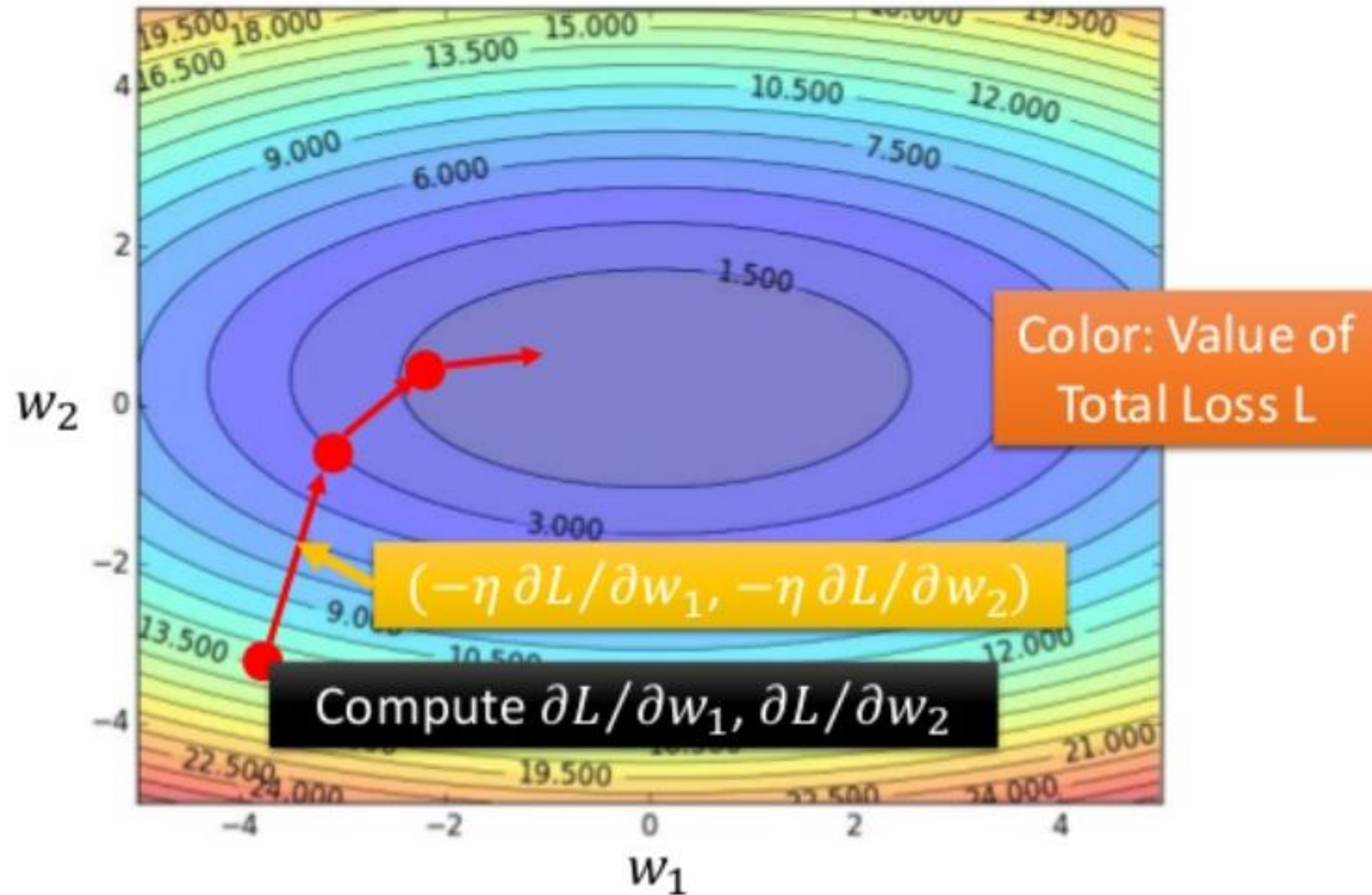
trained by error back-propagation algorithm (倒傳遞演算法). 由結果矯正前面參數

Gradient Descent



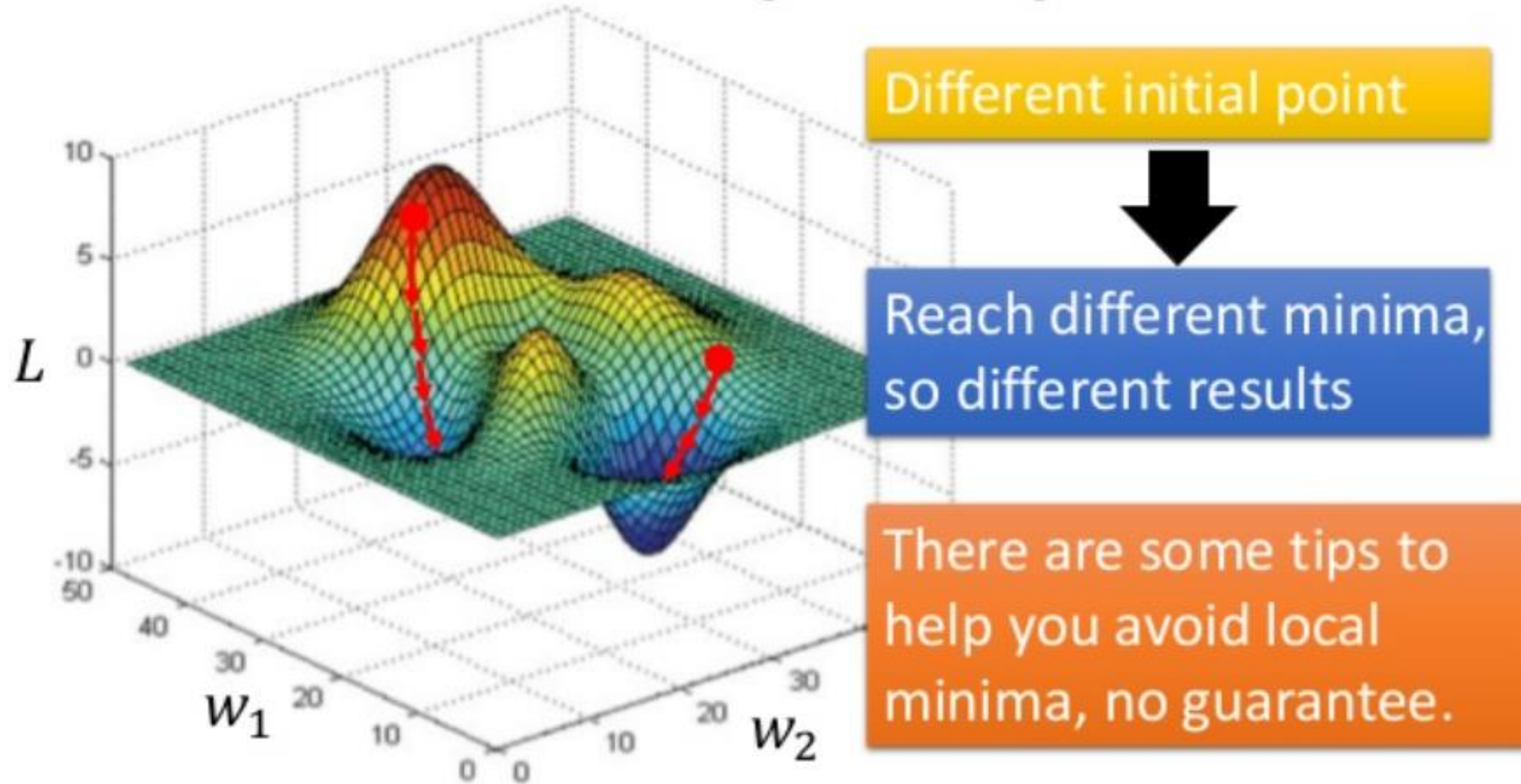
Gradient Descent

Hopfully, we would reach
a minima



Gradient Descent - Difficulty

- Gradient descent never guarantee global minima



Gradient Descent

This is the “learning” of machines in deep learning

➡ Even alpha go using this approach.

People image



Actually

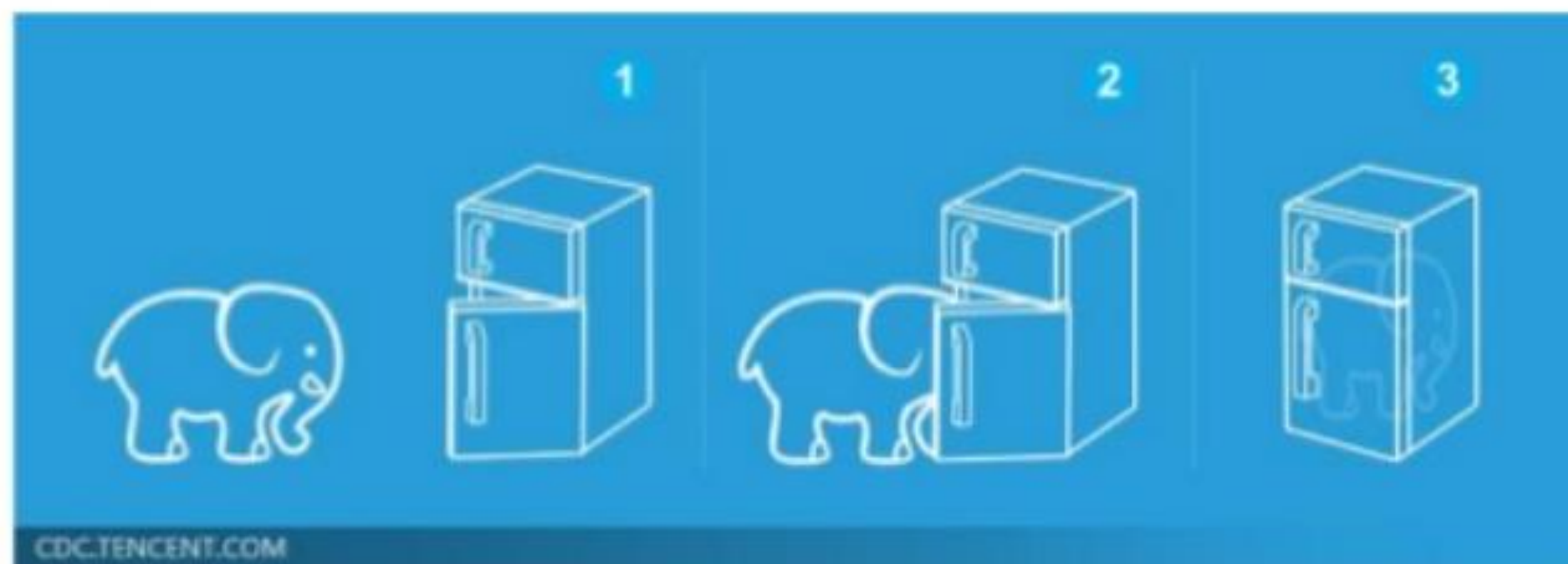


I hope you are not too disappointed :p

Concluding Remarks



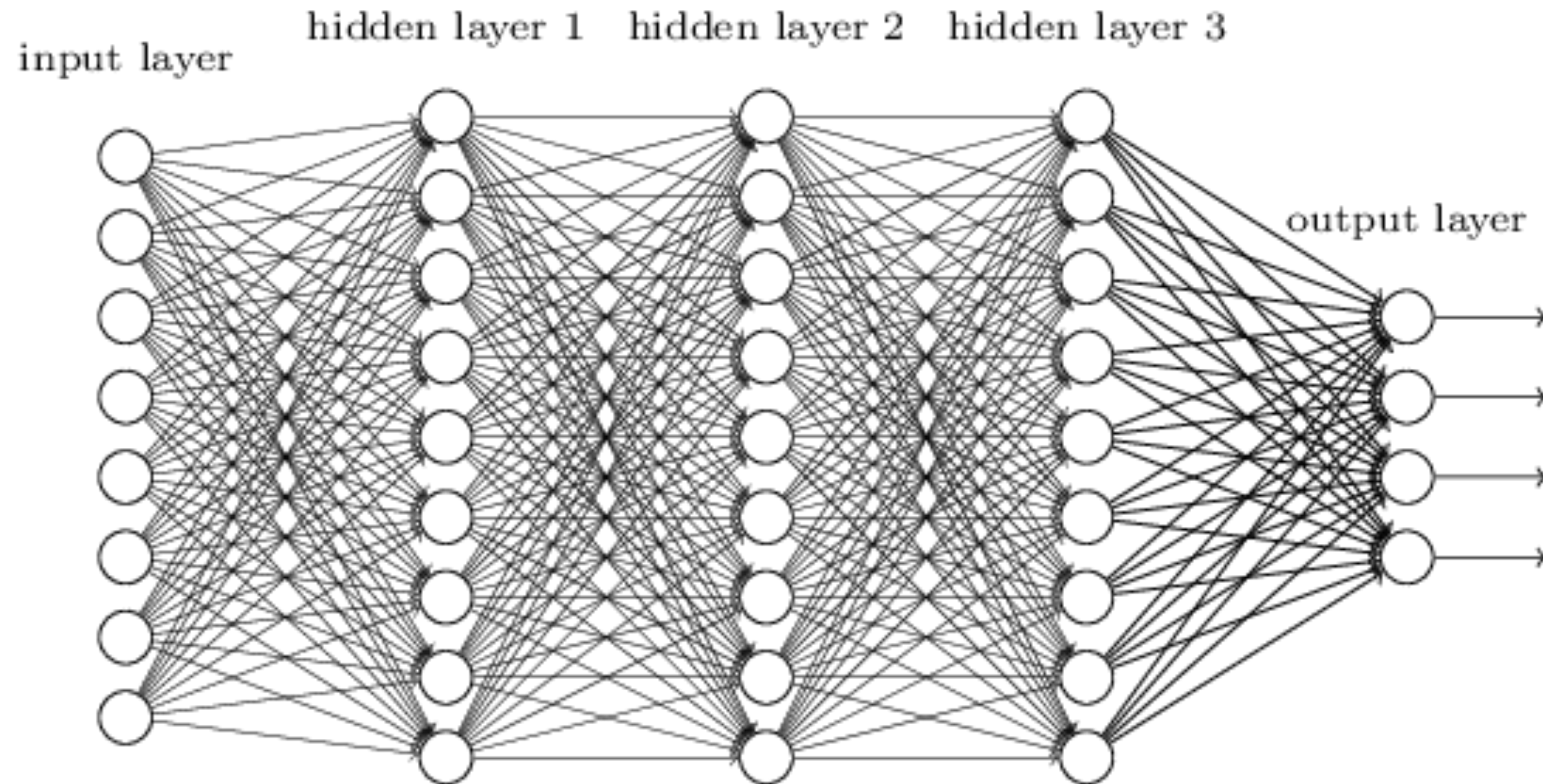
Deep Learning is so simple



Summary:

Training:

1. 設定參數。包括hidden layer(隱藏層)數目，隱藏層神經元數目、學習速率(learning rate)等。
2. Training 之目的在於學習在此網路設定下之權重值 (by back-propagation algorithm 倒傳遞演算法)。





NessLab

Overview of popular Deep Learning Techniques - CNN

- **Convolutional Neural Networks (CNNs)** 卷積神經網路

→ dimension reduction.

1. The convolutional layer acts as a feature extraction layer, and the fully-connected network acts as a classifiers. So, a CNN is actually a total solution for automatic pattern recognition model.
2. Originally designed for 2-D images (spatial data)
3. Can also applied to 1-D time-series signals (e.g., motor fault detection)
 - 1-D CNN may not be the best for recognizing 1-D signals → solution : digital signal processing techniques, such as short-time Fourier Transform (STFT) and wavelet transform (WT) can be utilized to transform 1-D signals to 2-D images (e.g., spectrograms). Then, feed them into 2-D CNNs

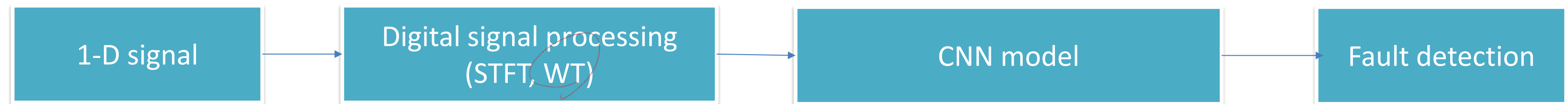


NessLab

Example: ML vs. CNN model for motor fault detection



Procedure of machine learning approach-based fault detection



Procedure of CNN-based for fault detection

Y.-M. Hsueh, V. R. Ittangihal, W.-B. Wu, H.-C. Chang, and C.-C. Kuo, "Fault diagnosis system for induction motors by CNN using empirical wavelet transform," *Symmetry*, vol. 11, no. 10, p. 1212, 2019.



NessLab

Overview of popular Deep Learning Techniques - RNN

- **Recurrent Neural Networks (RNNs) 遞迴 or 循環神經網路**
 1. Proposed in the early 1980s, and become popular in 2010
 2. An effective DL model for recognizing 1-D time series signals.
 3. RNNs are suited for temporal sequence data, because RNNs captures dynamic time-domain characteristics by looping outputs back into the networks as inputs for subsequent steps. This action enables RNNs to preserve information across different time points.
 4. Unlike CNNs, designed for 2D spatial data, RNNs are designed for 1-D signals.
 5. Long short-term memory (LSTM) (長短期記憶模型) is a commonly-used RNN.

H. C. Cho, J. Knowles, M. S. Fadali, and K. S. Lee, "Fault detection and isolation of induction motors using recurrent neural networks and dynamic Bayesian modeling," *IEEE Transactions on Control Systems Technology*, vol. 18, no. 2, pp. 430-437, 2009.